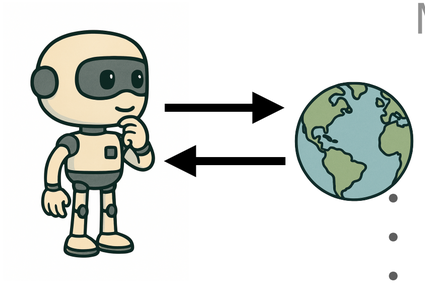
L13: Landmark Deep RL Algorithms Part 1

On-policy methods: CPI → TRPO → PPO

EECE 571N | Sequential Decision Making | Fall 2025

Cyrus Neary | cyrus.neary@ubc.ca

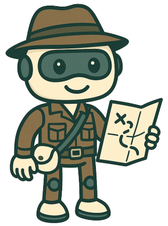
Where are we?

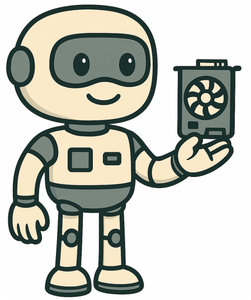


Part 1

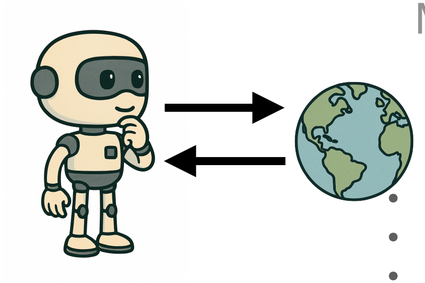
Modeling and solving sequential decision-making problems via dynamic programming.

- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs





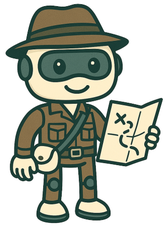
Where are we?



Part 1

Modeling and solving sequential decision-making problems via dynamic programming.

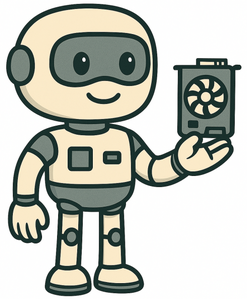
- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs



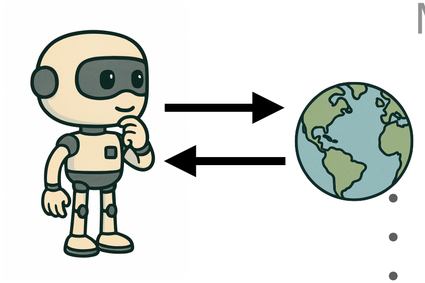
Part 2

The RL setting and fundamental RL algorithms.

- Exploration vs. Exploitation: Bandits algorithms
- Temporal differencing, Q-learning
- Model-based RL and sample complexity



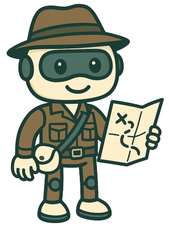
Where are we?



Part 1

Modeling and solving sequential decision-making problems via dynamic programming.

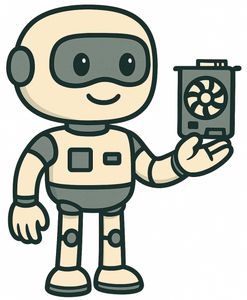
- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs



Part 2

The RL setting and fundamental RL algorithms.

- Exploration vs. Exploitation: Bandits algorithms
- Temporal differencing, Q-learning
- Model-based RL and sample complexity

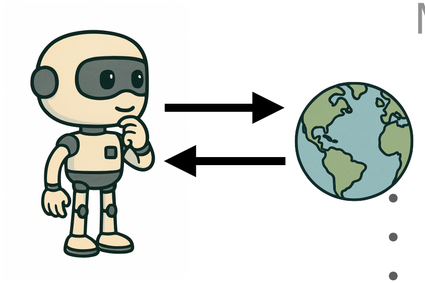


Part 3

Survey of deep RL algorithms.

- Function approximation
- Deep Q learning
- Challenges with deep RL
- Policy gradient algorithms
- Landmark deep RL algorithms

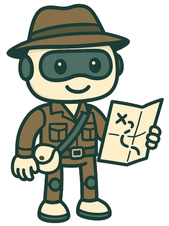
Where are we?



Part 1

Modeling and solving sequential decision-making problems via dynamic programming.

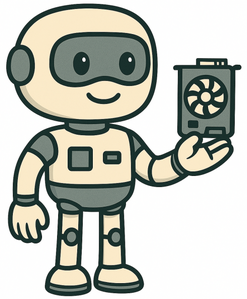
- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs



Part 2

The RL setting and fundamental RL algorithms.

- Exploration vs. Exploitation: Bandits algorithms
- Temporal differencing, Q-learning
- Model-based RL and sample complexity



Part 3

Survey of deep RL algorithms.

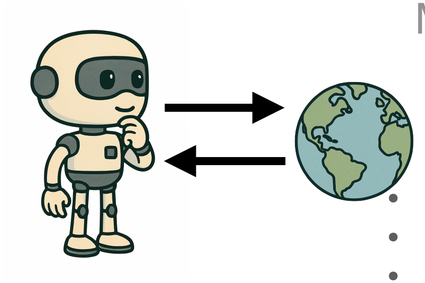
- Function approximation
- Deep Q learning
- Challenges with deep RL
- Policy gradient algorithms
- Landmark deep RL algorithms

Part 4

Advanced topics and current research.

- Intro to hierarchical RL
- Intro to multiagent decision-making
- Sim-to-real methods
- Behavioral cloning
- Generalist robotic agents
- Safe sequential decision-making

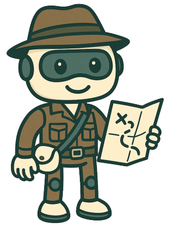
Where are we?



Part 1

Modeling and solving sequential decision-making problems via dynamic programming.

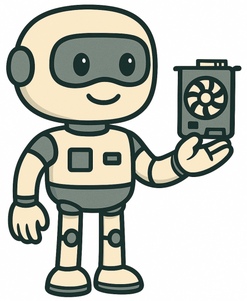
- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs



Part 2

The RL setting and fundamental RL algorithms.

- Exploration vs. Exploitation: Bandits algorithms
- Temporal differencing, Q-learning
- Model-based RL and sample complexity



Part 3

Survey of deep RL algorithms.

- Function approximation
- Deep Q learning
- Challenges with deep RL
- Policy gradient algorithms
- Landmark deep RL algorithms

Part 4

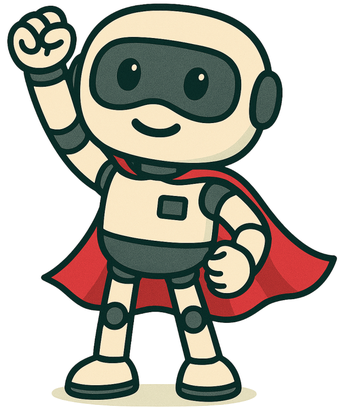
Advanced topics and current research.

- Intro to hierarchical RL
- Intro to multiagent decision-making
- Sim-to-real methods
- Behavioral cloning
- Generalist robotic agents
- Safe sequential decision-making

Last lecture: the policy gradient theorem

$$\nabla J(\theta) = \nabla V^{\pi}(s_0)$$

$$\propto \sum_{s \in S} \mu(s) \sum_{a \in A} Q_{\pi_\theta}(s,a) \nabla \pi_\theta(s,a)$$



Normalized expected state visitation frequency over an episode induced by policy π_θ.

Note: this part will be slightly different in different cases (episodic vs. Continuing, discounted vs undiscounted vs average reward).

True state-action value function of policy π_θ.

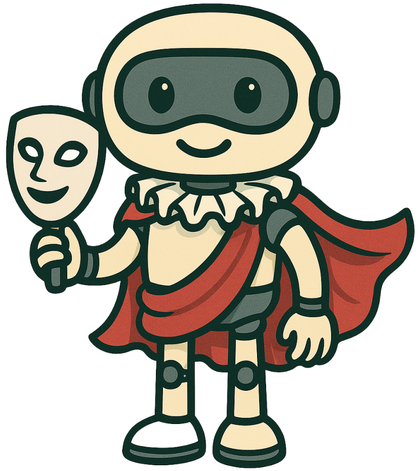
Crucially: $\nabla J(\theta)$ doesn't depend on the gradient through the environment dynamics!

Algorithmic implication: We can estimate $\nabla J(\theta) = \mathbb{E}_{S \sim \mu, A \sim \pi_\theta(\cdot|S)} \left[Q_{\pi_\theta}(S,A) \nabla log(\pi_\theta(A \mid S)) \right]$ by running π_θ in the environment, estimating $Q_{\pi_\theta}(s,a)$, and computing averages of samples of $Q^{\pi_\theta}(s^{(i)}, a^{(i)}) \nabla \log(\pi_\theta(s^{(i)}, a^{(i)}))$.

Actor-critic methods

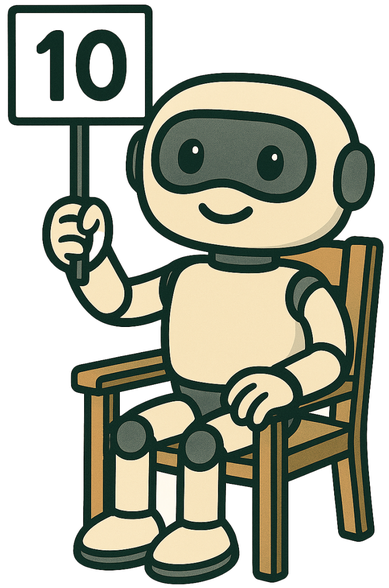
Generally speaking, RL algorithms that combine:

- An **actor**: the parameterized policy $\pi_\theta(a \mid s)$.
- A **critic**: an approximation of a value function $V_w(s)$ or $Q_w(s, a)$ that evaluates how good the actor's choices are.



Actor $\pi_\theta(a \mid s)$

Takes actions in the environment, given the current state.



Critic $V_w(s)$

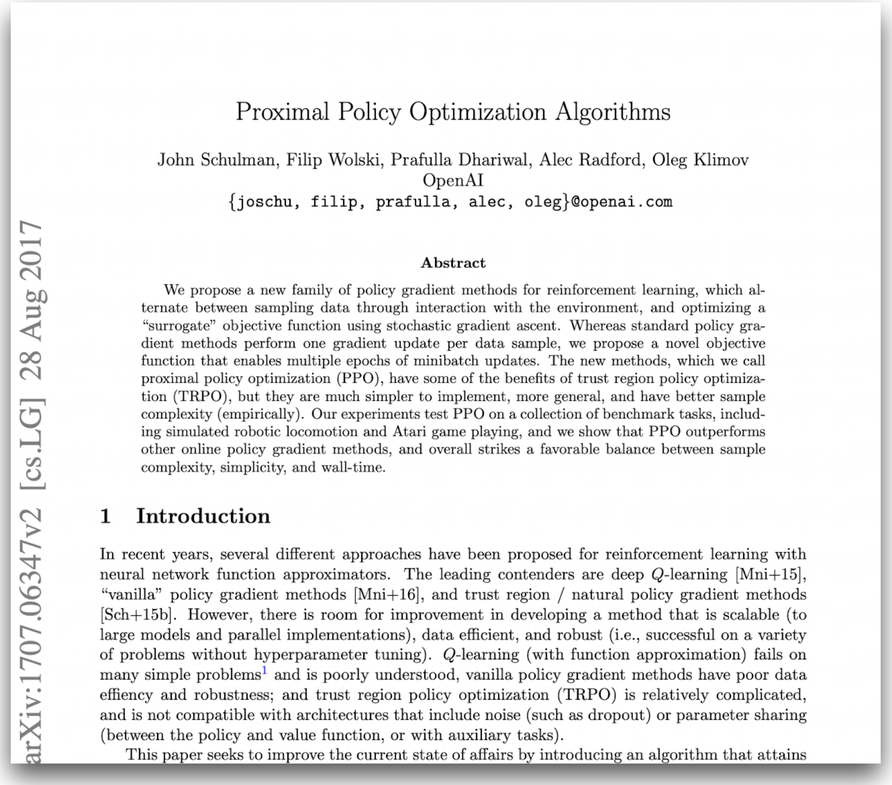
Evaluates the quality of the actor's chosen actions. Used to improve the actor's decisions during training.

Both the actor and the critic are learned simultaneously as the learning algorithm progresses.

Today: Proximal policy optimization

Strong algorithm, seems to work almost everywhere

Published in 2017, still a go-to algorithm in 2025.



Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

arXiv:1707.06347v2 [cs.LG] 28 Aug 2017

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a "surrogate" objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q-learning [Mni+15], "vanilla" policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q-learning (with function approximation) fails on many simple problems[1] and is poorly understood, vanilla policy gradient methods have poor data effiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

This paper seeks to improve the current state of affairs by introducing an algorithm that attains

Today: Proximal policy optimization

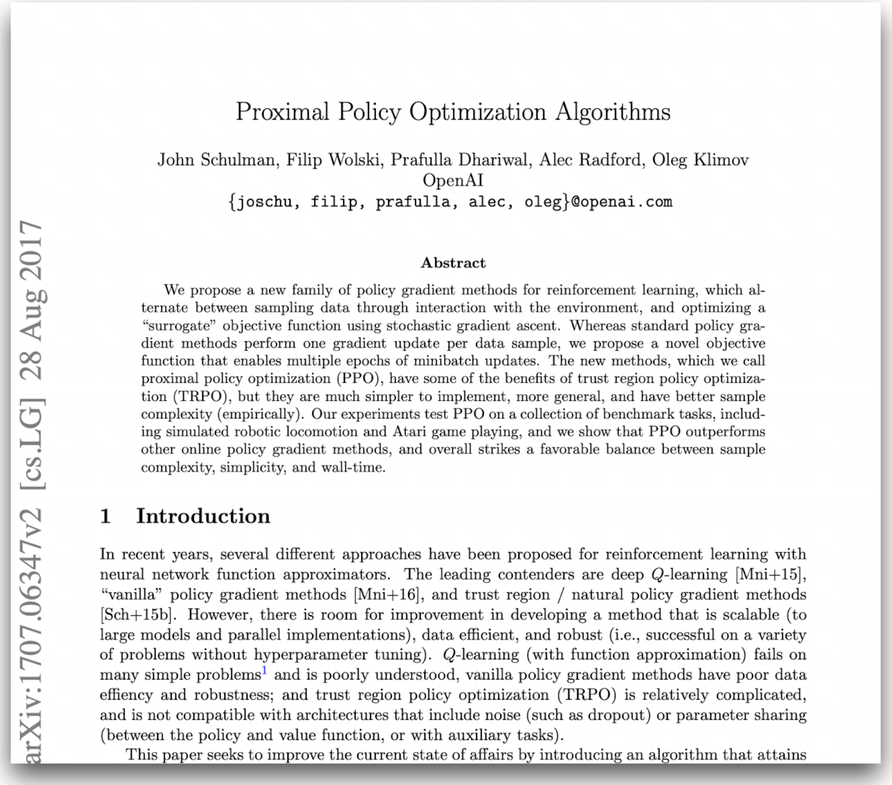
Strong algorithm, seems to work almost everywhere

Published in 2017, still a go-to algorithm in 2025.

So, how does it work?

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for



Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

arXiv:1707.06347v2 [cs.LG] 28 Aug 2017

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a "surrogate" objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q-learning [Mni+15], "vanilla" policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q-learning (with function approximation) fails on many simple problems[1] and is poorly understood, vanilla policy gradient methods have poor data effiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

This paper seeks to improve the current state of affairs by introducing an algorithm that attains

Today: Proximal policy optimization

Strong algorithm, seems to work almost everywhere

Published in 2017, still a go-to algorithm in 2025.

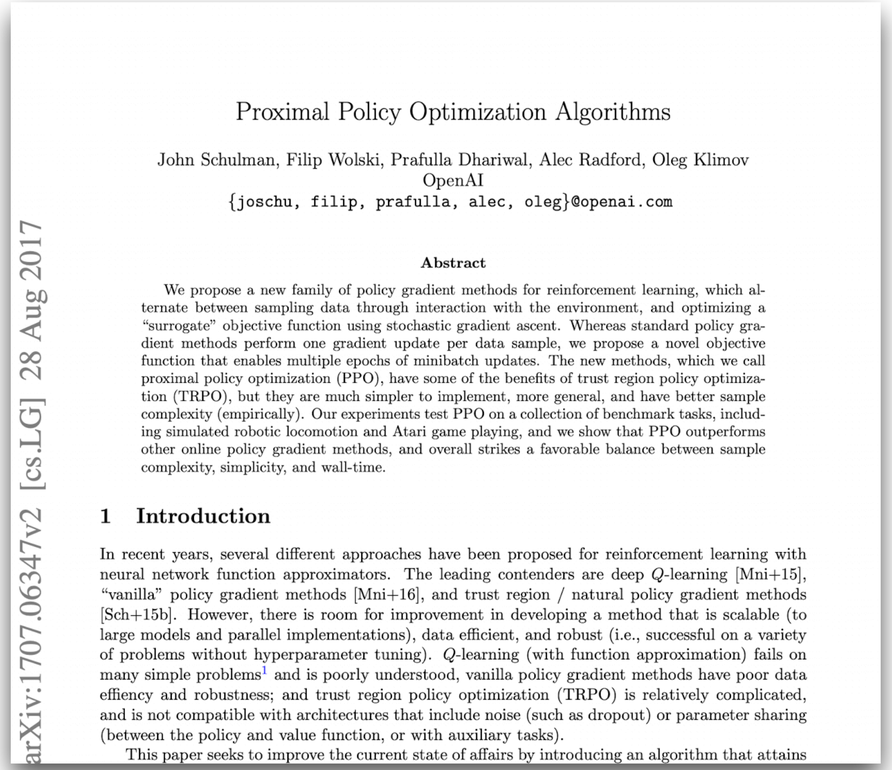
So, how does it work?

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

$$L_t^{PPO}(\theta) = \mathbb{E}_t\left[\min\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_\theta^{old}(a_t \mid s_t)}\hat{A}_t, \text{clip}\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_\theta^{old}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right]$$
$$-c_1\left(V_\theta(s_t) - V_t^{targ}\right)^2 + c_2 \text{entropy}(\pi_\theta(s_t))$$

Where $\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \ldots + \ldots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$,
and $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$



Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

arXiv:1707.06347v2 [cs.LG] 28 Aug 2017

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a "surrogate" objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep *Q*-learning [Mni+15], "vanilla" policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). *Q*-learning (with function approximation) fails on many simple problems[1] and is poorly understood, vanilla policy gradient methods have poor data effiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

This paper seeks to improve the current state of affairs by introducing an algorithm that attains

Today: Proximal policy optimization

Strong algorithm, seems to work almost everywhere

Published in 2017, still a go-to algorithm in 2025.

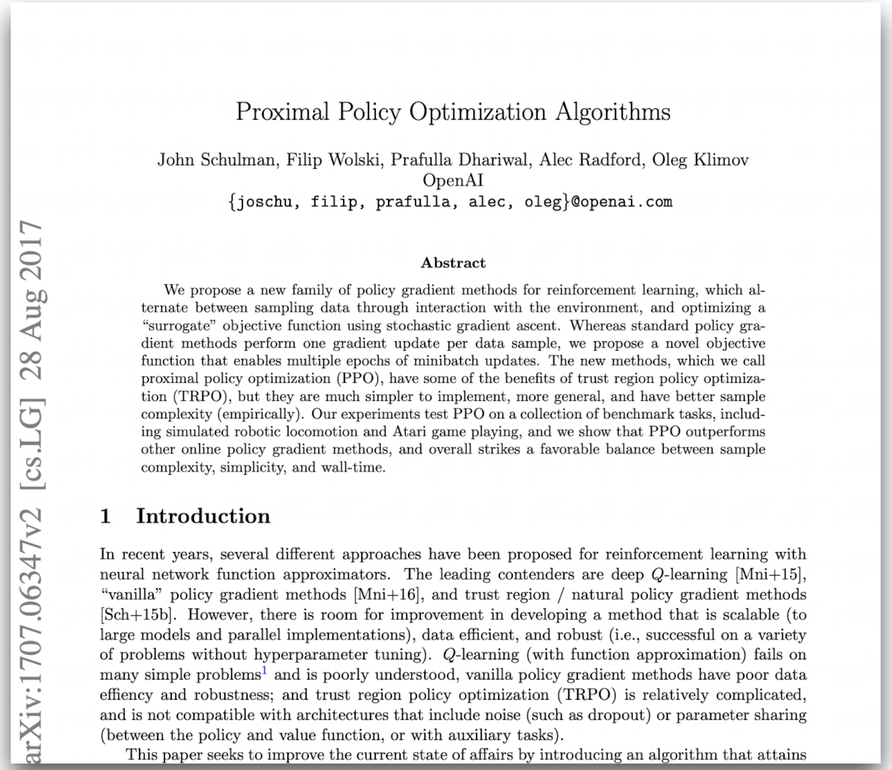
So, how does it work?

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

$$L_t^{PPO}(\theta) = \mathbb{E}_t\left[\min\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_\theta^{old}(a_t \mid s_t)}\hat{A}_t, \text{clip}\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_\theta^{old}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right]$$
$$-c_1\left(V_\theta(s_t) - V_t^{targ}\right)^2 + c_2 \text{entropy}(\pi_\theta(s_t))$$

Where $\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \ldots + \ldots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$,
and $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$



arXiv:1707.06347v2 [cs.LG] 28 Aug 2017

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a "surrogate" objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q-learning [Mni+15], "vanilla" policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q-learning (with function approximation) fails on many simple problems[1] and is poorly understood, vanilla policy gradient methods have poor data effiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

This paper seeks to improve the current state of affairs by introducing an algorithm that attains

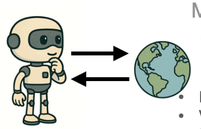
What? Why??

Its a long story…

The road to PPO

<u>Fundamentals of SDM and RL</u>

- Dynamic programming
- Generalized policy iteration
- Temporal difference learning
- Policy gradient methods



Part 1

Modeling and solving sequential decision-making problems via dynamic programming.

- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs

Part 2

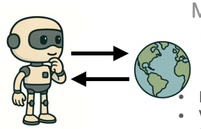
The RL setting and fundamental RL algorithms.

- Exploration vs. Exploitation: Bandits algorithms
- Temporal differencing, Q-learning
- Model-based RL and sample complexity

The road to PPO

Fundamentals of SDM and RL

- Dynamic programming
- Generalized policy iteration
- Temporal difference learning
- Policy gradient methods



Part 1

Modeling and solving sequential decision-making problems via dynamic programming.

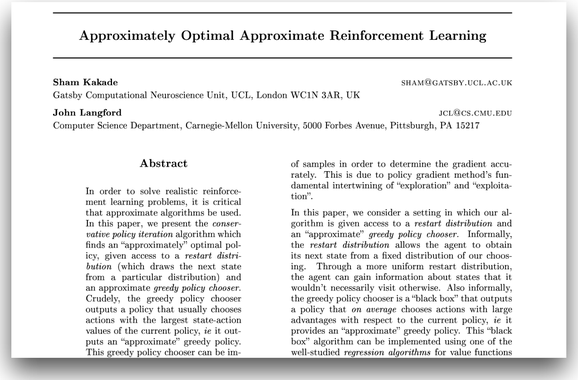
- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs

Part 2

The RL setting and fundamental RL algorithms.

- Exploration vs. Exploitation: Bandits algorithms
- Temporal differencing, Q-learning
- Model-based RL and sample complexity

Conservative policy iteration



Approximately Optimal Approximate Reinforcement Learning

Sham Kakade SHAM@GATSBY.UCL.AC.UK
Gatsby Computational Neuroscience Unit, UCL, London WC1N 3AR, UK
John Langford JCL@CS.CMU.EDU
Computer Science Department, Carnegie-Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15217

Abstract

In order to solve realistic reinforcement learning problems, it is critical that approximate algorithms be used. In this paper, we present the *conservative policy iteration* algorithm which finds an "approximately" optimal policy, given access to a *restart distribution* (which draws the next state from a particular distribution) and an approximate *greedy policy chooser*. Crudely, the greedy policy chooser outputs a policy that usually chooses actions with the largest state-action values of the current policy, *ie* it outputs an "approximate" greedy policy. This greedy policy chooser can be implemented using one of the well-studied *regression algorithms* for value functions of samples in order to determine the gradient accurately. This is due to policy gradient method's fundamental intertwining of "exploration" and "exploitation".

In this paper, we consider a setting in which our algorithm is given access to a *restart distribution* and an "approximate" *greedy policy chooser*. Informally, the *restart distribution* allows the agent to obtain its next state from a fixed distribution of our choosing. Through a more uniform restart distribution, the agent can gain information about states that it wouldn't necessarily visit otherwise. Also informally, the greedy policy chooser is a "black box" that outputs a policy that *on average* chooses actions with large advantages with respect to the current policy, *ie* it provides an "approximate" greedy policy. This "black box" algorithm can be implemented using one of the well-studied *regression algorithms* for value functions

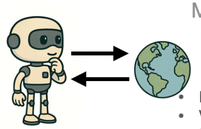
What happens to our policy improvement steps when we use imperfect, sample-based estimates of $Q^{\pi}(s, a)$?

How do still guarantee monotonic policy improvements in this setting?

The road to PPO

Fundamentals of SDM and RL

- Dynamic programming
- Generalized policy iteration
- Temporal difference learning
- Policy gradient methods



Part 1

Modeling and solving sequential decision-making problems via dynamic programming.

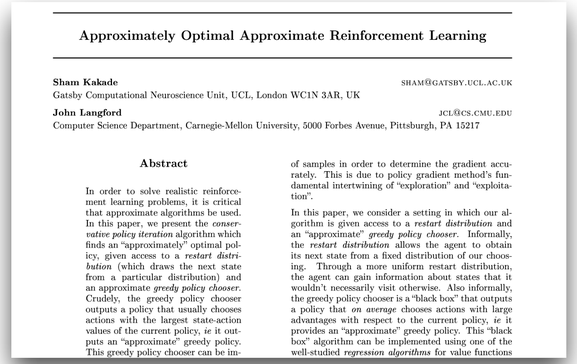
- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs

Part 2

The RL setting and fundamental RL algorithms.

- Exploration vs. Exploitation: Bandits algorithms
- Temporal differencing, Q-learning
- Model-based RL and sample complexity

Conservative policy iteration



Approximately Optimal Approximate Reinforcement Learning

Sham Kakade SHAM@GATSBY.UCL.AC.UK
Gatsby Computational Neuroscience Unit, UCL, London WC1N 3AR, UK

John Langford JCL@CS.CMU.EDU
Computer Science Department, Carnegie-Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15217

Abstract

In order to solve realistic reinforcement learning problems, it is critical that approximate algorithms be used. In this paper, we present the *conservative policy iteration* algorithm which finds an "approximately" optimal policy, given access to a *restart distribution* (which draws the next state from a particular distribution) and an approximate *greedy policy chooser*. Crudely, the greedy policy chooser outputs a policy that usually chooses actions with the largest state-action values of the current policy, *ie* it outputs an "approximate" greedy policy. This greedy policy chooser can be implemented using one of the well-studied *regression algorithms* for value functions of samples in order to determine the gradient accurately. This is due to policy gradient method's fundamental intertwining of "exploration" and "exploitation".

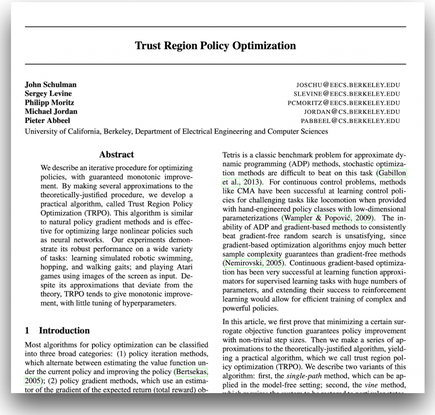
In this paper, we consider a setting in which our algorithm is given access to a *restart distribution* and an "approximate" *greedy policy chooser*. Informally, the *restart distribution* allows the agent to obtain its next state from a fixed distribution of our choosing. Through a more uniform restart distribution, the agent can gain information about states that it wouldn't necessarily visit otherwise. Also informally, the greedy policy chooser is a "black box" that outputs a policy that *on average* chooses actions with large advantages with respect to the current policy, *ie* it provides an "approximate" greedy policy. This "black box" algorithm can be implemented using one of the well-studied *regression algorithms* for value functions

What happens to our policy improvement steps when we use imperfect, sample-based estimates of $Q^{\pi}(s, a)$?

How do still guarantee monotonic policy improvements in this setting?

Trust region policy optimization

- Generalizes results,
- Principled optimization approach
- Practical deep RL algorithm with strong empirical results.



Trust Region Policy Optimization

John Schulman JOSCHU@EECS.BERKELEY.EDU
Sergey Levine SLEVINE@EECS.BERKELEY.EDU
Philipp Moritz PCMORITZ@EECS.BERKELEY.EDU
Michael Jordan JORDAN@CS.BERKELEY.EDU
Pieter Abbeel PABBEEL@CS.BERKELEY.EDU
University of California, Berkeley, Department of Electrical Engineering and Computer Sciences

Abstract

We describe an iterative procedure for optimizing policies, with guaranteed monotonic improvement. By making several approximations to the theoretically-justified procedure, we develop a practical algorithm, called Trust Region Policy Optimization (TRPO). This algorithm is similar to natural policy gradient methods and is effective for optimizing large nonlinear policies such as neural networks. Our experiments demonstrate its robust performance on a wide variety of tasks: learning simulated robotic swimming, hopping, and walking gaits; and playing Atari games using images of the screen as input. Despite its approximations that deviate from the theory, TRPO tends to give monotonic improvement, with little tuning of hyperparameters.

1 Introduction

Most algorithms for policy optimization can be classified into three broad categories: (1) policy iteration methods, which alternate between estimating the value function under the current policy and improving the policy (Bertsekas, 2005); (2) policy gradient methods, which use an estimator of the gradient of the expected return (total reward) ob-

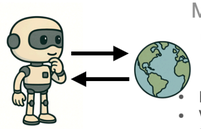
Tetris is a classic benchmark problem for approximate dynamic programming (ADP) methods, stochastic optimization methods are difficult to beat on this task (Gabillon et al., 2013). For continuous control problems, methods like CMA have been successful at learning control policies for challenging tasks like locomotion when provided with hand-engineered policy classes with low-dimensional parameterizations (Wampler & Popović, 2009). The inability of ADP and gradient-based methods to consistently beat gradient-free random search is unsatisfying, since gradient-based optimization algorithms enjoy much better sample complexity guarantees than gradient-free methods (Nemirovski, 2005). Continuous gradient-based optimization has been very successful at learning function approximators for supervised learning tasks with huge numbers of parameters, and extending their success to reinforcement learning would allow for efficient training of complex and powerful policies.

In this article, we first prove that minimizing a certain surrogate objective function guarantees policy improvement with non-trivial step sizes. Then we make a series of approximations to the theoretically-justified algorithm, yielding a practical algorithm, which we call trust region policy optimization (TRPO). We describe two variants of this algorithm: first, the *single-path* method, which can be applied in the model-free setting; second, the *vine* method,

The road to PPO

Fundamentals of SDM and RL

- Dynamic programming
- Generalized policy iteration
- Temporal difference learning
- Policy gradient methods



Part 1

Modeling and solving sequential decision-making problems via dynamic programming.

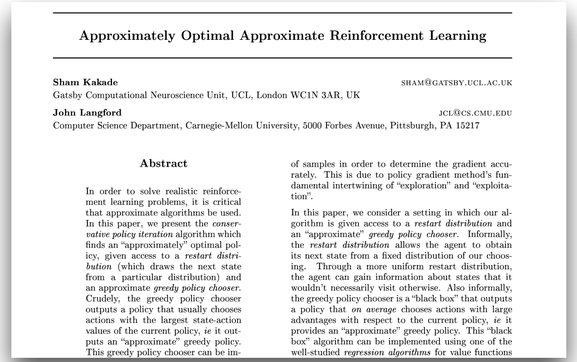
- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs

Part 2

The RL setting and fundamental RL algorithms.

- Exploration vs. Exploitation: Bandits algorithms
- Temporal differencing, Q-learning
- Model-based RL and sample complexity

Conservative policy iteration



Approximately Optimal Approximate Reinforcement Learning

Sham Kakade — SHAM@GATSBY.UCL.AC.UK
Gatsby Computational Neuroscience Unit, UCL, London WC1N 3AR, UK

John Langford — JCL@CS.CMU.EDU
Computer Science Department, Carnegie-Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15217

Abstract

In order to solve realistic reinforcement learning problems, it is critical that approximate algorithms be used. In this paper, we present the *conservative policy iteration* algorithm which finds an "approximately" optimal policy, given access to a *restart distribution* (which draws the next state from a particular distribution) and an approximate *greedy policy chooser*. Crudely, the greedy policy chooser outputs a policy that usually chooses actions with the largest state-action values of the current policy, ie it outputs an "approximate" greedy policy. This greedy policy chooser can be im-

of samples in order to determine the gradient accurately. This is due to policy gradient method's fundamental intertwining of "exploration" and "exploitation".

In this paper, we consider a setting in which our algorithm is given access to a *restart distribution* and an "approximate" *greedy policy chooser*. Informally, the *restart distribution* allows the agent to obtain its next state from a fixed distribution of our choosing. Through a more uniform restart distribution, the agent can gain information about states that it wouldn't necessarily visit otherwise. Also informally, the greedy policy chooser is a "black box" that outputs a policy that *on average* chooses actions with large advantages with respect to the current policy, ie it provides an "approximate" greedy policy. This "black box" algorithm can be implemented using one of the well-studied *regression algorithms* for value functions

What happens to our policy improvement steps when we use imperfect, sample-based estimates of $Q^{\pi}(s,a)$?

How do still guarantee monotonic policy improvements in this setting?

Trust region policy optimization

- Generalizes results,
- Principled optimization approach
- Practical deep RL algorithm with strong empirical results.

Trust Region Policy Optimization

John Schulman — JOSCHU@EECS.BERKELEY.EDU
Sergey Levine — SLEVINE@EECS.BERKELEY.EDU
Philipp Moritz — PCMORITZ@EECS.BERKELEY.EDU
Michael Jordan — JORDAN@CS.BERKELEY.EDU
Pieter Abbeel — PABBEEL@CS.BERKELEY.EDU
University of California, Berkeley, Department of Electrical Engineering and Computer Sciences

Abstract

We describe an iterative procedure for optimizing policies, with guaranteed monotonic improvement. By making several approximations to the theoretically-justified procedure, we develop a practical algorithm, called Trust Region Policy Optimization (TRPO). This algorithm is similar to natural policy gradient methods and is effective for optimizing large nonlinear policies such as neural networks. Our experiments demonstrate its robust performance on a wide variety of tasks: learning simulated robotic swimming, hopping, and walking gaits; and playing Atari games using images of the screen as input. Despite its approximations that deviate from the theory, TRPO tends to give monotonic improvement, with little tuning of hyperparameters.

1 Introduction

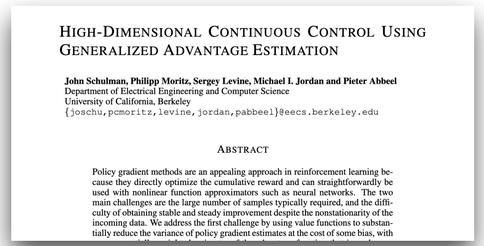
Most algorithms for policy optimization can be classified into three broad categories: (1) policy iteration methods, which alternate between estimating the value function under the current policy and improving the policy (Bertsekas, 2005); (2) policy gradient methods, which use an estimator of the gradient of the expected return (total reward) ob-

Tetris is a classic benchmark problem for approximate dynamic programming (ADP) methods, stochastic optimization methods are difficult to beat on this task (Gabillon et al., 2013). For continuous control problems, methods like CMA have been successful at learning control policies for challenging tasks like locomotion when provided with hand-engineered policy classes with low-dimensional parameterizations (Wampler & Popović, 2009). The inability of ADP and gradient-based methods to consistently beat gradient-free random search is unsatisfying, since gradient-based optimization algorithms enjoy much better sample complexity guarantees than gradient-free methods (Nemirovski, 2005). Continuous gradient-based optimization has been very successful at learning function approximators for supervised learning tasks with huge numbers of parameters, and extending their success to reinforcement learning would allow for efficient training of complex and powerful policies.

In this article, we first prove that minimizing a certain surrogate objective function guarantees policy improvement with non-trivial step sizes. Then we make a series of approximations to the theoretically-justified algorithm, yielding a practical algorithm, which we call trust region policy optimization (TRPO). We describe two variants of this algorithm: first, the *single-path* method, which can be applied in the model-free setting; second, the *vine* method,

Generalized Avantage Estimation

Improved low-variance, bias-tuneable advantage function estimation.



HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel
Department of Electrical Engineering and Computer Science
University of California, Berkeley
{joschu,pcmoritz,levine,jordan,pabbeel}@eecs.berkeley.edu

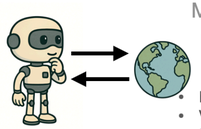
ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with

The road to PPO

Fundamentals of SDM and RL

- Dynamic programming
- Generalized policy iteration
- Temporal difference learning
- Policy gradient methods



Part 1

Modeling and solving sequential decision-making problems via dynamic programming.

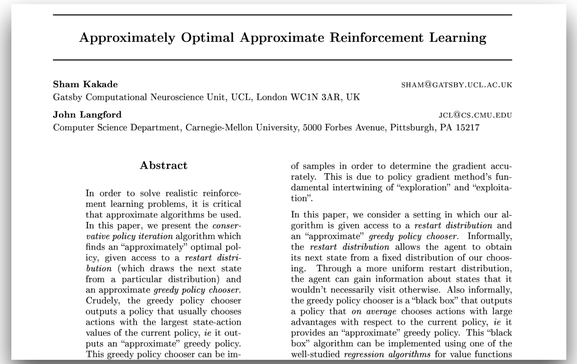
- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs

Part 2

The RL setting and fundamental RL algorithms.

- Exploration vs. Exploitation: Bandits algorithms
- Temporal differencing, Q-learning
- Model-based RL and sample complexity

Conservative policy iteration



Approximately Optimal Approximate Reinforcement Learning

Sham Kakade — SHAM@GATSBY.UCL.AC.UK
Gatsby Computational Neuroscience Unit, UCL, London WC1N 3AR, UK

John Langford — JCL@CS.CMU.EDU
Computer Science Department, Carnegie-Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15217

Abstract

In order to solve realistic reinforcement learning problems, it is critical that approximate algorithms be used. In this paper, we present the *conservative policy iteration* algorithm which finds an "approximately" optimal policy, given access to a *restart distribution* (which draws the next state from a particular distribution) and an approximate *greedy policy chooser*. Crudely, the greedy policy chooser outputs a policy that usually chooses actions with the largest state-action values of the current policy, *ie* it outputs an "approximate" greedy policy. This greedy policy chooser can be implemented using one of the well-studied *regression algorithms* for value functions of samples in order to determine the gradient accurately. This is due to policy gradient method's fundamental intertwining of "exploration" and "exploitation".

In this paper, we consider a setting in which our algorithm is given access to a *restart distribution* and an "approximate" *greedy policy chooser*. Informally, the *restart distribution* allows the agent to obtain its next state from a fixed distribution of our choosing. Through a more uniform restart distribution, the agent can gain information about states that it wouldn't necessarily visit otherwise. Also informally, the greedy policy chooser is a "black box" that outputs a policy that *on average* chooses actions with large advantages with respect to the current policy, *ie* it provides an "approximate" greedy policy. This "black box" algorithm can be implemented using one of the well-studied *regression algorithms* for value functions

What happens to our policy improvement steps when we use imperfect, sample-based estimates of $Q^{\pi}(s, a)$?

How do still guarantee monotonic policy improvements in this setting?

Trust region policy optimization

- Generalizes results,
- Principled optimization approach
- Practical deep RL algorithm with strong empirical results.

Trust Region Policy Optimization

John Schulman — JOSCHU@EECS.BERKELEY.EDU
Sergey Levine — SLEVINE@EECS.BERKELEY.EDU
Philipp Moritz — PCMORITZ@EECS.BERKELEY.EDU
Michael Jordan — JORDAN@CS.BERKELEY.EDU
Pieter Abbeel — PABBEEL@CS.BERKELEY.EDU
University of California, Berkeley, Department of Electrical Engineering and Computer Sciences

Abstract

We describe an iterative procedure for optimizing policies, with guaranteed monotonic improvement. By making several approximations to the theoretically-justified procedure, we develop a practical algorithm, called Trust Region Policy Optimization (TRPO). This algorithm is similar to natural policy gradient methods and is effective for optimizing large nonlinear policies such as neural networks. Our experiments demonstrate its robust performance on a wide variety of tasks: learning simulated robotic swimming, hopping, and walking gaits; and playing Atari games using images of the screen as input. Despite its approximations that deviate from the theory, TRPO tends to give monotonic improvement, with little tuning of hyperparameters.

1 Introduction

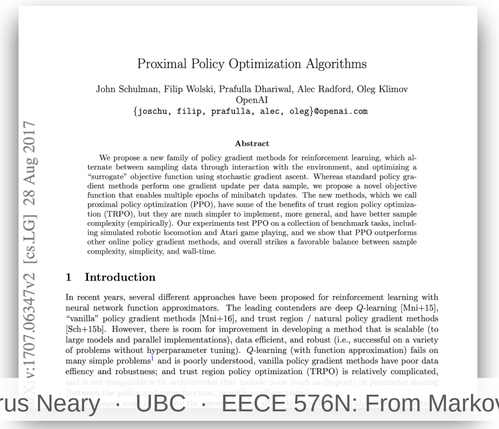
Most algorithms for policy optimization can be classified into three broad categories: (1) policy iteration methods, which alternate between estimating the value function under the current policy and improving the policy (Bertsekas, 2005); (2) policy gradient methods, which use an estimator of the gradient of the expected return (total reward) ob-

Tetris is a classic benchmark problem for approximate dynamic programming (ADP) methods, stochastic optimization methods are difficult to beat on this task (Gabillon et al., 2013). For continuous control problems, methods like CMA have been successful at learning control policies for challenging tasks like locomotion when provided with hand-engineered policy classes with low-dimensional parameterizations (Wampler & Popović, 2009). The inability of ADP and gradient-based methods to consistently beat gradient-free random search is unsatisfying, since gradient-based optimization algorithms enjoy much better sample complexity guarantees than gradient-free methods (Nemirovski, 2005). Continuous gradient-based optimization has been very successful at learning function approximators for supervised learning tasks with huge numbers of parameters, and extending their success to reinforcement learning would allow for efficient training of complex and powerful policies.

In this article, we first prove that minimizing a certain surrogate objective function guarantees policy improvement with non-trivial step sizes. Then we make a series of approximations to the theoretically-justified algorithm, yielding a practical algorithm, which we call trust region policy optimization (TRPO). We describe two variants of this algorithm: first, the *single-path* method, which can be applied in the model-free setting; second, the *vine* method,

Proximal policy optimization

Simplification of TRPO's constrained optimization problem into a clipped surrogate objective.



Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a "surrogate" objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

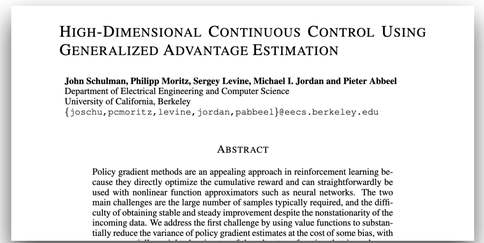
1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q-learning [Mni+15], "vanilla" policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q-learning (with function approximation) fails on many simple problems[1] and is poorly understood, vanilla policy gradient methods have poor data effiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing

arXiv:1707.06347v2 [cs.LG] 28 Aug 2017

Generalized Avantage Estimation

Improved low-variance, bias-tuneable advantage function estimation.



High-Dimensional Continuous Control Using Generalized Advantage Estimation

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel
Department of Electrical Engineering and Computer Science
University of California, Berkeley
{joschu,pcmoritz,levine,jordan,pabbeel}@eecs.berkeley.edu

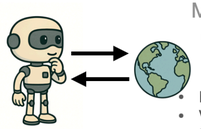
Abstract

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with

The road to PPO

Fundamentals of SDM and RL

- Dynamic programming
- Generalized policy iteration
- Temporal difference learning
- Policy gradient methods



Part 1

Modeling and solving sequential decision-making problems via dynamic programming.

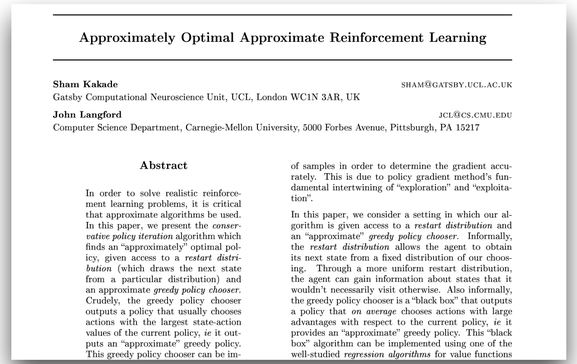
- Markov Decision Processes
- Value and Policy Iteration
- LP solutions
- Connections with optimal control
- POMDPs

Part 2

The RL setting and fundamental RL algorithms.

- Exploration vs. Exploitation: Bandits algorithms
- Temporal differencing, Q-learning
- Model-based RL and sample complexity

Conservative policy iteration



Approximately Optimal Approximate Reinforcement Learning

Sham Kakade — SHAM@GATSBY.UCL.AC.UK
Gatsby Computational Neuroscience Unit, UCL, London WC1N 3AR, UK

John Langford — JCL@CS.CMU.EDU
Computer Science Department, Carnegie-Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15217

Abstract

In order to solve realistic reinforcement learning problems, it is critical that approximate algorithms be used. In this paper, we present the *conservative policy iteration* algorithm which finds an "approximately" optimal policy, given access to a *restart distribution* (which draws the next state from a particular distribution) and an approximate *greedy policy chooser*. Crudely, the greedy policy chooser outputs a policy that usually chooses actions with the largest state-action values of the current policy, ie it outputs an "approximate" greedy policy. This greedy policy chooser can be im-

of samples in order to determine the gradient accurately. This is due to policy gradient method's fundamental intertwining of "exploration" and "exploitation".

In this paper, we consider a setting in which our algorithm is given access to a *restart distribution* and an "approximate" *greedy policy chooser*. Informally, the *restart distribution* allows the agent to obtain its next state from a fixed distribution of our choosing. Through a more uniform restart distribution, the agent can gain information about states that it wouldn't necessarily visit otherwise. Also informally, the greedy policy chooser is a "black box" that outputs a policy that *on average* chooses actions with large advantages with respect to the current policy, ie it provides an "approximate" greedy policy. This "black box" algorithm can be implemented using one of the well-studied *regression algorithms* for value functions

What happens to our policy improvement steps when we use imperfect, sample-based estimates of $Q^{\pi}(s, a)$?

How do still guarantee monotonic policy improvements in this setting?

Trust region policy optimization

- Generalizes results,
- Principled optimization approach
- Practical deep RL algorithm with strong empirical results.

Trust Region Policy Optimization

John Schulman — JOSCHU@EECS.BERKELEY.EDU
Sergey Levine — SLEVINE@EECS.BERKELEY.EDU
Philipp Moritz — PCMORITZ@EECS.BERKELEY.EDU
Michael Jordan — JORDAN@CS.BERKELEY.EDU
Pieter Abbeel — PABBEEL@CS.BERKELEY.EDU
University of California, Berkeley, Department of Electrical Engineering and Computer Sciences

Abstract

We describe an iterative procedure for optimizing policies, with guaranteed monotonic improvement. By making several approximations to the theoretically-justified procedure, we develop a practical algorithm, called Trust Region Policy Optimization (TRPO). This algorithm is similar to natural policy gradient methods and is effective for optimizing large nonlinear policies such as neural networks. Our experiments demonstrate its robust performance on a wide variety of tasks: learning simulated robotic swimming, hopping, and walking gaits; and playing Atari games using images of the screen as input. Despite its approximations that deviate from the theory, TRPO tends to give monotonic improvement, with little tuning of hyperparameters.

1 Introduction

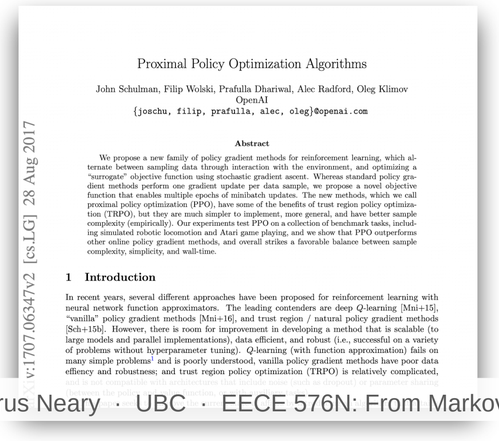
Most algorithms for policy optimization can be classified into three broad categories: (1) policy iteration methods, which alternate between estimating the value function under the current policy and improving the policy (Bertsekas, 2005); (2) policy gradient methods, which use an estimator of the gradient of the expected return (total reward) ob-

Tetris is a classic benchmark problem for approximate dynamic programming (ADP) methods, stochastic optimization methods are difficult to beat on this task (Gabillon et al., 2013). For continuous control problems, methods like CMA have been successful at learning control policies for challenging tasks like locomotion when provided with hand-engineered policy classes with low-dimensional parameterizations (Wampler & Popović, 2009). The inability of ADP and gradient-based methods to consistently beat gradient-free random search is unsatisfying, since gradient-based optimization algorithms enjoy much better sample complexity guarantees than gradient-free methods (Nemirovski, 2005). Continuous gradient-based optimization has been very successful at learning function approximators for supervised learning tasks with huge numbers of parameters, and extending their success to reinforcement learning would allow for efficient training of complex and powerful policies.

In this article, we first prove that minimizing a certain surrogate objective function guarantees policy improvement with non-trivial step sizes. Then we make a series of approximations to the theoretically-justified algorithm, yielding a practical algorithm, which we call trust region policy optimization (TRPO). We describe two variants of this algorithm: first, the *single-path* method, which can be applied in the model-free setting; second, the *vine* method,

Proximal policy optimization

Simplification of TRPO's constrained optimization problem into a clipped surrogate objective.



Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a "surrogate" objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

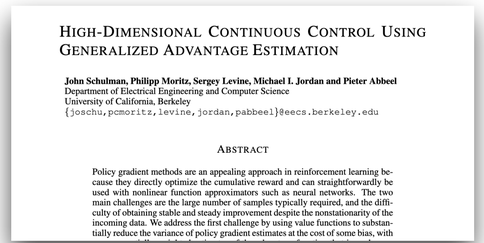
1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q-learning [Mni+15], "vanilla" policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q-learning (with function approximation) fails on many simple problems[1] and is poorly understood, vanilla policy gradient methods have poor data effiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing

arXiv:1707.06347v2 [cs.LG] 28 Aug 2017

Generalized Avantage Estimation

Improved low-variance, bias-tuneable advantage function estimation.



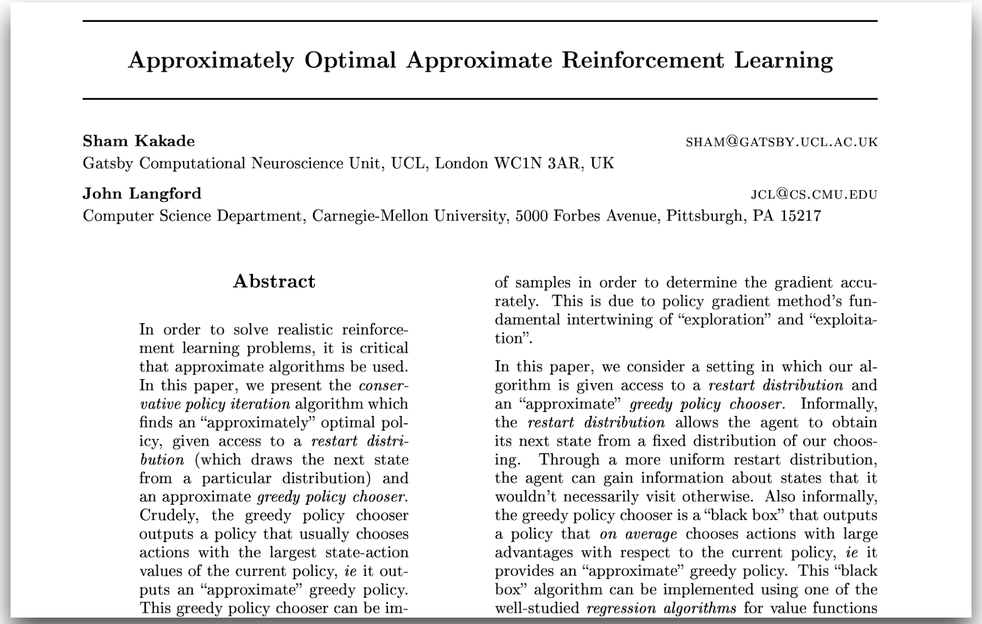
HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel
Department of Electrical Engineering and Computer Science
University of California, Berkeley
{joschu,pcmoritz,levine,jordan,pabbeel}@eecs.berkeley.edu

ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with

Conservative policy iteration



Approximately Optimal Approximate Reinforcement Learning

Sham Kakade SHAM@GATSBY.UCL.AC.UK
Gatsby Computational Neuroscience Unit, UCL, London WC1N 3AR, UK

John Langford JCL@CS.CMU.EDU
Computer Science Department, Carnegie-Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15217

Abstract

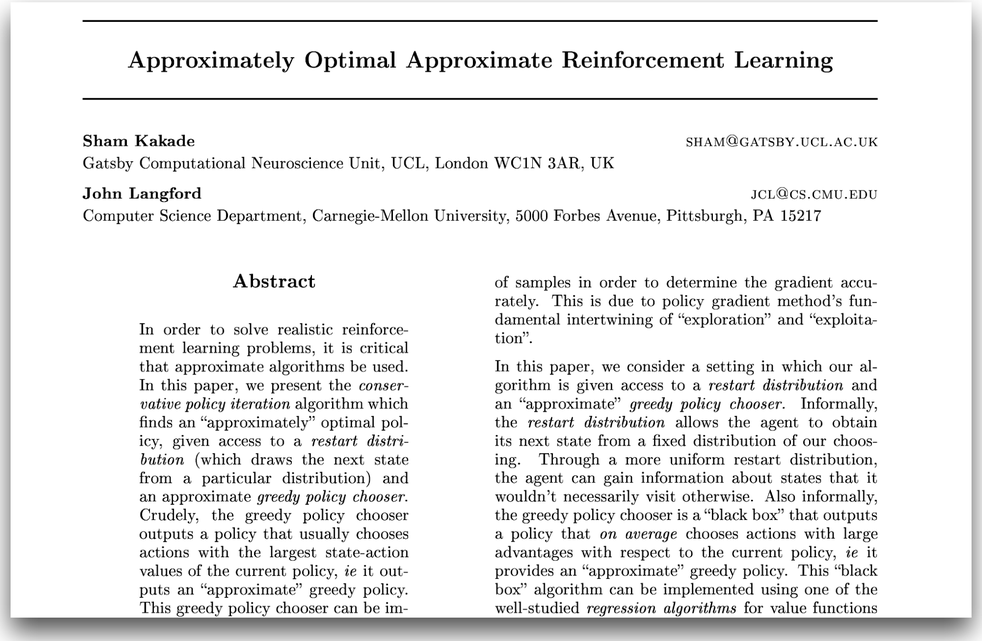
In order to solve realistic reinforcement learning problems, it is critical that approximate algorithms be used. In this paper, we present the *conservative policy iteration* algorithm which finds an "approximately" optimal policy, given access to a *restart distribution* (which draws the next state from a particular distribution) and an approximate *greedy policy chooser*. Crudely, the greedy policy chooser outputs a policy that usually chooses actions with the largest state-action values of the current policy, *ie* it outputs an "approximate" greedy policy. This greedy policy chooser can be implemented using one of the well-studied *regression algorithms* for value functions of samples in order to determine the gradient accurately. This is due to policy gradient method's fundamental intertwining of "exploration" and "exploitation".

In this paper, we consider a setting in which our algorithm is given access to a *restart distribution* and an "approximate" *greedy policy chooser*. Informally, the *restart distribution* allows the agent to obtain its next state from a fixed distribution of our choosing. Through a more uniform restart distribution, the agent can gain information about states that it wouldn't necessarily visit otherwise. Also informally, the greedy policy chooser is a "black box" that outputs a policy that *on average* chooses actions with large advantages with respect to the current policy, *ie* it provides an "approximate" greedy policy. This "black box" algorithm can be implemented using one of the well-studied *regression algorithms* for value functions

A primarily theoretical contribution asking the following core questions:

When policy evaluation and improvement are both approximate, how can we design updates that still guarantee monotonic improvement in true expected return?

Conservative policy iteration



Approximately Optimal Approximate Reinforcement Learning

Sham Kakade SHAM@GATSBY.UCL.AC.UK
Gatsby Computational Neuroscience Unit, UCL, London WC1N 3AR, UK

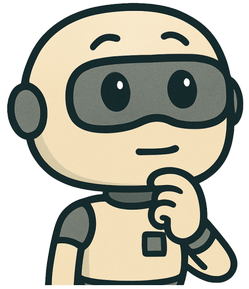
John Langford JCL@CS.CMU.EDU
Computer Science Department, Carnegie-Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15217

Abstract

In order to solve realistic reinforcement learning problems, it is critical that approximate algorithms be used. In this paper, we present the *conservative policy iteration* algorithm which finds an "approximately" optimal policy, given access to a *restart distribution* (which draws the next state from a particular distribution) and an approximate *greedy policy chooser*. Crudely, the greedy policy chooser outputs a policy that usually chooses actions with the largest state-action values of the current policy, *ie* it outputs an "approximate" greedy policy. This greedy policy chooser can be implemented using one of the well-studied *regression algorithms* for value functions of samples in order to determine the gradient accurately. This is due to policy gradient method's fundamental intertwining of "exploration" and "exploitation".

In this paper, we consider a setting in which our algorithm is given access to a *restart distribution* and an "approximate" *greedy policy chooser*. Informally, the *restart distribution* allows the agent to obtain its next state from a fixed distribution of our choosing. Through a more uniform restart distribution, the agent can gain information about states that it wouldn't necessarily visit otherwise. Also informally, the greedy policy chooser is a "black box" that outputs a policy that *on average* chooses actions with large advantages with respect to the current policy, *ie* it provides an "approximate" greedy policy. This "black box" algorithm can be implemented using one of the well-studied *regression algorithms* for value functions

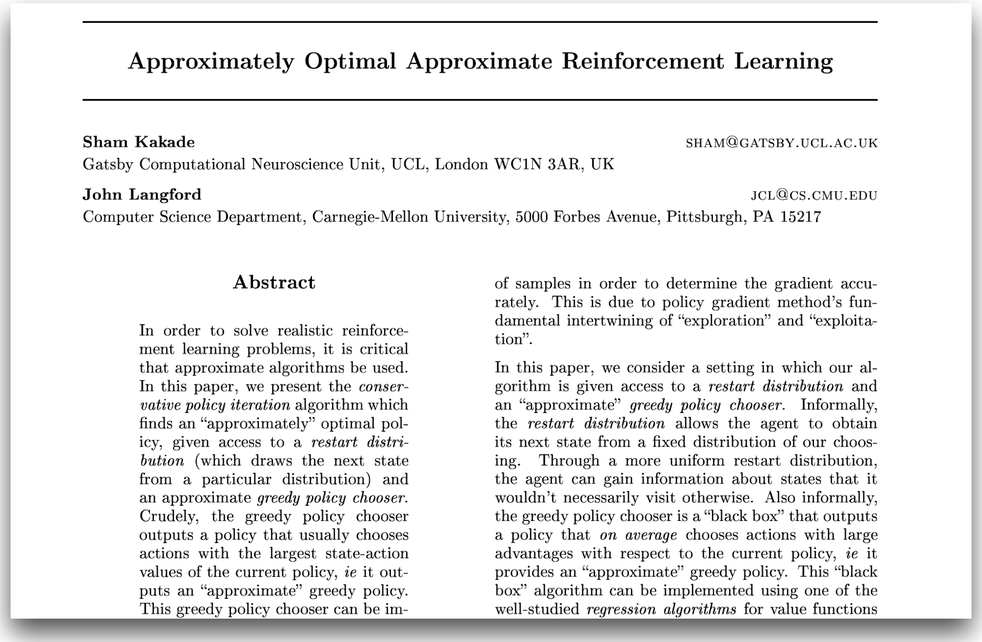
A primarily theoretical contribution asking the following core questions:
When policy evaluation and improvement are both approximate, how can we design updates that still guarantee monotonic improvement in true expected return?



Policy iteration guarantees improvement only under exact evaluation and exact greedy improvement.

In practice, RL relies on approximate value functions, finite sampling, and function approximation, so both of these steps can be noisy or biased breaking the guarantee.

Conservative policy iteration



Approximately Optimal Approximate Reinforcement Learning

Sham Kakade SHAM@GATSBY.UCL.AC.UK
Gatsby Computational Neuroscience Unit, UCL, London WC1N 3AR, UK

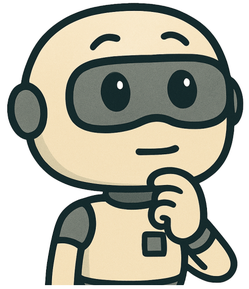
John Langford JCL@CS.CMU.EDU
Computer Science Department, Carnegie-Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15217

Abstract

In order to solve realistic reinforcement learning problems, it is critical that approximate algorithms be used. In this paper, we present the *conservative policy iteration* algorithm which finds an "approximately" optimal policy, given access to a *restart distribution* (which draws the next state from a particular distribution) and an approximate *greedy policy chooser*. Crudely, the greedy policy chooser outputs a policy that usually chooses actions with the largest state-action values of the current policy, *ie* it outputs an "approximate" greedy policy. This greedy policy chooser can be implemented using one of the well-studied *regression algorithms* for value functions of samples in order to determine the gradient accurately. This is due to policy gradient method's fundamental intertwining of "exploration" and "exploitation".

In this paper, we consider a setting in which our algorithm is given access to a *restart distribution* and an "approximate" *greedy policy chooser*. Informally, the *restart distribution* allows the agent to obtain its next state from a fixed distribution of our choosing. Through a more uniform restart distribution, the agent can gain information about states that it wouldn't necessarily visit otherwise. Also informally, the greedy policy chooser is a "black box" that outputs a policy that *on average* chooses actions with large advantages with respect to the current policy, *ie* it provides an "approximate" greedy policy. This "black box" algorithm can be implemented using one of the well-studied *regression algorithms* for value functions

A primarily theoretical contribution asking the following core questions:
When policy evaluation and improvement are both approximate, how can we design updates that still guarantee monotonic improvement in true expected return?

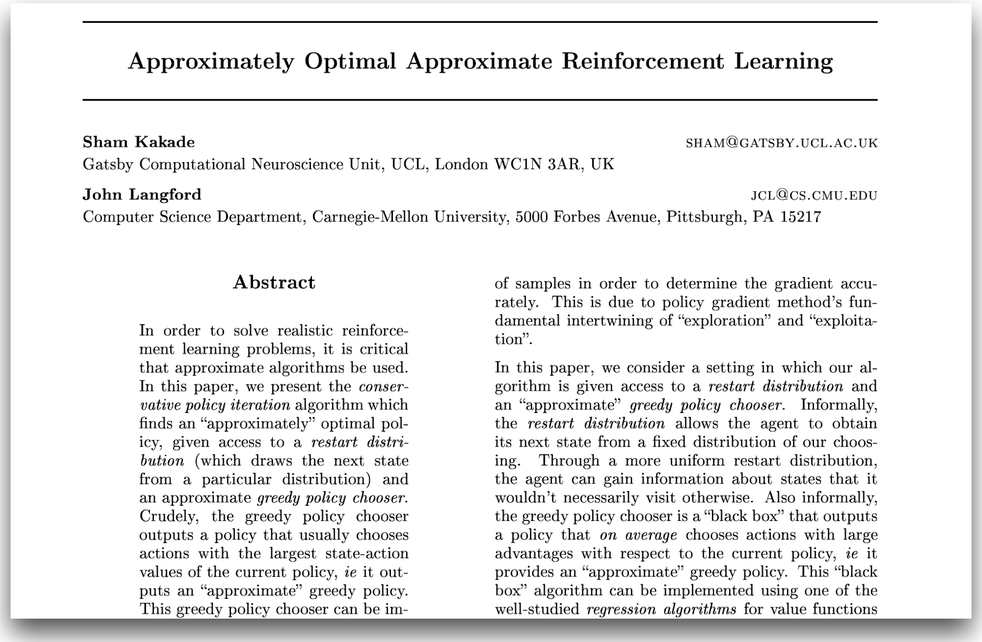


Policy iteration guarantees improvement only under exact evaluation and exact greedy improvement.

In practice, RL relies on approximate value functions, finite sampling, and function approximation, so both of these steps can be noisy or biased breaking the guarantee.

Key takeaway: You can't trust greedy policy updates when value estimates are approximate, but you *can* guarantee monotonic improvement if you make the update small enough.

Conservative policy iteration



Approximately Optimal Approximate Reinforcement Learning

Sham Kakade SHAM@GATSBY.UCL.AC.UK
Gatsby Computational Neuroscience Unit, UCL, London WC1N 3AR, UK

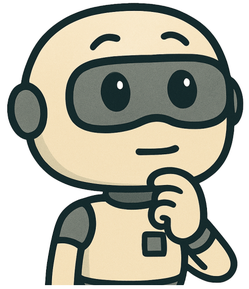
John Langford JCL@CS.CMU.EDU
Computer Science Department, Carnegie-Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15217

Abstract

In order to solve realistic reinforcement learning problems, it is critical that approximate algorithms be used. In this paper, we present the *conservative policy iteration* algorithm which finds an "approximately" optimal policy, given access to a *restart distribution* (which draws the next state from a particular distribution) and an approximate *greedy policy chooser*. Crudely, the greedy policy chooser outputs a policy that usually chooses actions with the largest state-action values of the current policy, *ie* it outputs an "approximate" greedy policy. This greedy policy chooser can be implemented using one of the well-studied *regression algorithms* for value functions of samples in order to determine the gradient accurately. This is due to policy gradient method's fundamental intertwining of "exploration" and "exploitation".

In this paper, we consider a setting in which our algorithm is given access to a *restart distribution* and an "approximate" *greedy policy chooser*. Informally, the *restart distribution* allows the agent to obtain its next state from a fixed distribution of our choosing. Through a more uniform restart distribution, the agent can gain information about states that it wouldn't necessarily visit otherwise. Also informally, the greedy policy chooser is a "black box" that outputs a policy that *on average* chooses actions with large advantages with respect to the current policy, *ie* it provides an "approximate" greedy policy. This "black box" algorithm can be implemented using one of the well-studied *regression algorithms* for value functions

<u>A primarily theoretical contribution asking the following core questions:</u>
When policy evaluation and improvement are both approximate, how can we design updates that still guarantee monotonic improvement in true expected return?



Policy iteration guarantees improvement only under exact evaluation and exact greedy improvement.

In practice, RL relies on approximate value functions, finite sampling, and function approximation, so both of these steps can be noisy or biased breaking the guarantee.

<u>Key takeaway:</u> You can't trust greedy policy updates when value estimates are approximate, but you *can* guarantee monotonic improvement if you make the update small enough.

We now argue that both greedy dynamic programming and policy gradient methods give unsatisfactory answers to these questions. Note that we did not ask is "What is the quality of the asymptotic policy?". We are only interested in policies that we can find in a reasonable amount of time. Understanding the problems with these current methods gives insight into our new algorithm, which addresses these three questions.

Conservative policy iteration

Approximately Optimal Approximate Reinforcement Learning

Sham Kakade SHAM@GATSBY.UCL.AC.UK
Gatsby Computational Neuroscience Unit, UCL, London WC1N 3AR, UK

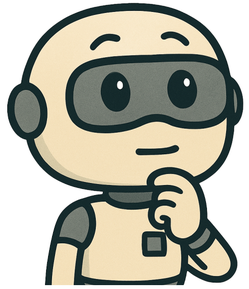
John Langford JCL@CS.CMU.EDU
Computer Science Department, Carnegie-Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15217

Abstract

In order to solve realistic reinforcement learning problems, it is critical that approximate algorithms be used. In this paper, we present the *conservative policy iteration* algorithm which finds an "approximately" optimal policy, given access to a *restart distribution* (which draws the next state from a particular distribution) and an approximate *greedy policy chooser*. Crudely, the greedy policy chooser outputs a policy that usually chooses actions with the largest state-action values of the current policy, *ie* it outputs an "approximate" greedy policy. This greedy policy chooser can be implemented using one of the well-studied *regression algorithms* for value functions of samples in order to determine the gradient accurately. This is due to policy gradient method's fundamental intertwining of "exploration" and "exploitation".

In this paper, we consider a setting in which our algorithm is given access to a *restart distribution* and an "approximate" *greedy policy chooser*. Informally, the *restart distribution* allows the agent to obtain its next state from a fixed distribution of our choosing. Through a more uniform restart distribution, the agent can gain information about states that it wouldn't necessarily visit otherwise. Also informally, the greedy policy chooser is a "black box" that outputs a policy that *on average* chooses actions with large advantages with respect to the current policy, *ie* it provides an "approximate" greedy policy. This "black box" algorithm can be implemented using one of the well-studied *regression algorithms* for value functions

A primarily theoretical contribution asking the following core questions:
When policy evaluation and improvement are both approximate, how can we design updates that still guarantee monotonic improvement in true expected return?



Policy iteration guarantees improvement only under exact evaluation and exact greedy improvement.

In practice, RL relies on approximate value functions, finite sampling, and function approximation, so both of these steps can be noisy or biased breaking the guarantee.

Key takeaway: You can't trust greedy policy updates when value estimates are approximate, but you *can* guarantee monotonic improvement if you make the update small enough.

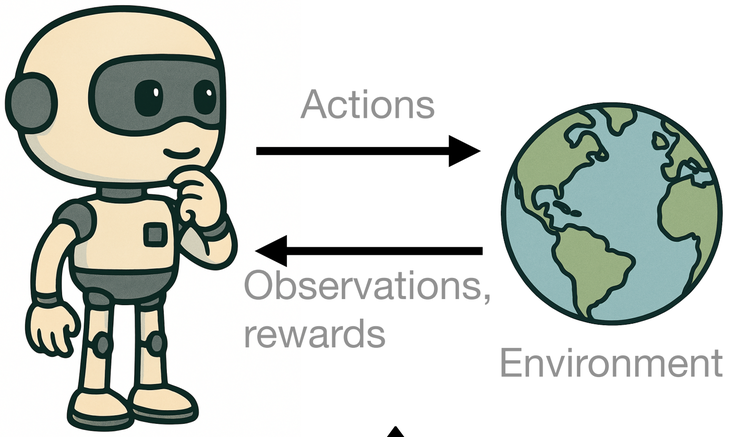
3 The Problems with Current Methods

We now examine in more detail the problems with approximate value function methods and policy gradient methods. There are three questions to which we desire answers to:

(1) Is there some performance measure that is guaranteed to improve at every step?
(2) How difficult is it to verify if a particular update improves this measure?
(3) After a reasonable number of policy updates, what performance level is obtained?

We now argue that both greedy dynamic programming and policy gradient methods give unsatisfactory answers to these questions. Note that we did not ask is "What is the quality of the asymptotic policy?". We are only interested in policies that we can find in a reasonable amount of time. Understanding the problems with these current methods gives insight into our new algorithm, which addresses these three questions.

Recall: Generalized policy iteration



$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

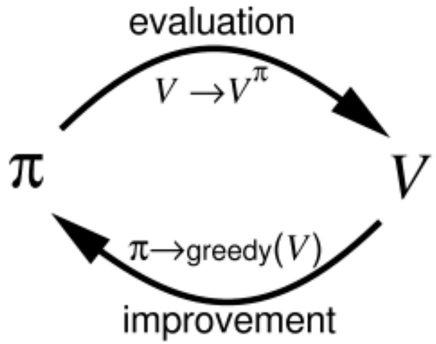


Figure courtesy of Sutton & Barto (2018)

Use estimate of $Q_\theta(s, a)$ to improve $\pi \to \pi'$

Learned parameterized estimate $Q_\theta(s, a)$

Advantage functions

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

A policy's advantage function is defined as the difference between its state-action value and state value.

Intuition:

- $Q^{\pi}(s,a)$ is the value of taking a specific action from state s, counter to the action that the policy might perscribe.
- $V^{\pi}(s)$ is the value at s, if π is followed immediately
- $A^{\pi}(s,a)$ is the difference in value that results from taking action a instead of following π for one step.

PG estimator with baseline subtraction

$$\theta_{k+1} = \theta_k + \alpha \left(Q^{\pi_\theta}(s,a) - V_w^{\pi_\theta}(S_t) \right) \nabla \ln \left(\pi_\theta(A_t \mid S_t) \right)$$

- we re-write that in terms of this newly defined advantage function

$$\theta_{k+1} = \theta_k + \alpha A^{\pi_\theta}(s,a) \nabla \ln \left(\pi_\theta(A_t \mid S_t) \right)$$

Policy advantage

Given two policies π, and π', it can be useful to reason about how much π' improves performance over π.

Policy advantage

Given two policies π, and π', it can be useful to reason about how much π' improves performance over π.

Ideally, we would have $A^{\pi'}(s, a) \geq A^{\pi}(s, a)$ for all state-action pairs (s, a).

If this were the case, then π' would be a strict improvement over π.

However, when π' is a greedy policy w.r.t. a **learned** value function, we cannot guarantee that $A^{\pi'}(s, a) > A^{\pi}(s, a),\ \forall (s, a)$.

Policy advantage

Given two policies π, and π', it can be useful to reason about how much π' improves performance over π.

Ideally, we would have $A^{\pi'}(s,a) \geq A^{\pi}(s,a)$ for all state-action pairs (s,a).

If this were the case, then π' would be a strict improvement over π.

However, when π' is a greedy policy w.r.t. a **learned** value function, we cannot guarantee that $A^{\pi'}(s,a) > A^{\pi}(s,a),\ \forall (s,a)$.

Instead, it can be helpful to reason about advantage improvements averaged over the initial policy's state distribution:

Policy advantage

Given two policies π, and π', it can be useful to reason about how much π' improves performance over π.

Ideally, we would have $A^{\pi'}(s,a) \geq A^{\pi}(s,a)$ for all state-action pairs (s,a).

If this were the case, then π' would be a strict improvement over π.

However, when π' is a greedy policy w.r.t. a **<u>learned</u>** value function, we cannot guarantee that $A^{\pi'}(s,a) > A^{\pi}(s,a),\ \forall (s,a)$.

Instead, it can be helpful to reason about advantage improvements averaged over the initial policy's state distribution:

Advantage of policy π' over policy π, from initial state distribution ρ.

$$\mathbb{A}^{\pi,\rho}(\pi') = \mathbb{E}_{s\sim d_{\pi,\rho}}\left[\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^{\pi}(s,a)\right]\right]$$

Policy advantage

Given two policies π, and π', it can be useful to reason about how much π' improves performance over π.

Ideally, we would have $A^{\pi'}(s,a) \geq A^{\pi}(s,a)$ for all state-action pairs (s,a).

If this were the case, then π' would be a strict improvement over π.

However, when π' is a greedy policy w.r.t. a **learned** value function, we cannot guarantee that $A^{\pi'}(s,a) > A^{\pi}(s,a),\ \forall (s,a)$.

Instead, it can be helpful to reason about advantage improvements averaged over the initial policy's state distribution:

Advantage of policy π' over policy π, from initial state distribution μ.

$$\mathbb{A}^{\pi,\rho}(\pi') = \mathbb{E}_{s\sim d_{\pi,\rho}}\left[\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^{\pi}(s,a)\right]\right]$$

State distribution under policy π executed from initial state distribution ρ.

Policy advantage

Given two policies π, and π', it can be useful to reason about how much π' improves performance over π.

Ideally, we would have $A^{\pi'}(s,a) \geq A^{\pi}(s,a)$ for all state-action pairs (s,a).

If this were the case, then π' would be a strict improvement over π.

However, when π' is a greedy policy w.r.t. a **learned** value function, we cannot guarantee that $A^{\pi'}(s,a) > A^{\pi}(s,a),\ \forall (s,a)$.

Instead, it can be helpful to reason about advantage improvements averaged over the initial policy's state distribution:

Advantage of policy π' over policy π, from initial state distribution μ.

$$\mathbb{A}^{\pi,\rho}(\pi') = \mathbb{E}_{s\sim d_{\pi,\rho}}\left[\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^{\pi}(s,a)\right]\right]$$

State distribution under policy π executed from initial state distribution ρ.

Actions drawn from the distribution of policy π'.

Policy advantage

Given two policies π, and π', it can be useful to reason about how much π' improves performance over π.

Ideally, we would have $A^{\pi'}(s,a) \geq A^{\pi}(s,a)$ for all state-action pairs (s,a).

If this were the case, then π' would be a strict improvement over π.

However, when π' is a greedy policy w.r.t. a **learned** value function, we cannot guarantee that $A^{\pi'}(s,a) > A^{\pi}(s,a),\ \forall (s,a)$.

Instead, it can be helpful to reason about advantage improvements averaged over the initial policy's state distribution:

Advantage of policy π' over policy π, from initial state distribution μ.

$$\mathbb{A}^{\pi,\rho}(\pi') = \mathbb{E}_{s\sim d_{\pi,\rho}}\left[\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^{\pi}(s,a)\right]\right]$$

State distribution under policy π executed from initial state distribution ρ.

Actions drawn from the distribution of policy π'.

Advantage function of policy π.

Policy advantage

Given two policies π, and π', it can be useful to reason about how much π' improves performance over π.

Ideally, we would have $A^{\pi'}(s,a) \geq A^{\pi}(s,a)$ for all state-action pairs (s,a).

If this were the case, then π' would be a strict improvement over π.

However, when π' is a greedy policy w.r.t. a **learned** value function, we cannot guarantee that $A^{\pi'}(s,a) > A^{\pi}(s,a),\ \forall (s,a)$.

Instead, it can be helpful to reason about advantage improvements averaged over the initial policy's state distribution:

Advantage of policy π' over policy π, from initial state distribution ρ.

$$\mathbb{A}^{\pi,\rho}(\pi') = \mathbb{E}_{s\sim d_{\pi,\rho}}\left[\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^{\pi}(s,a)\right]\right]$$

State distribution under policy π executed from initial state distribution ρ.

Actions drawn from the distribution of policy π'.

Advantage function of policy π.

"The policy advantage measures the degree to which π' is choosing actions with large advantages, with respect to the set of states visited under π starting from a state $s \sim \rho$."

Theorem 4.1, Kakade & Langford 2002

Suppose π' is a greedy policy with respect to some approximate state-action values $\hat{A}^{\pi}(s,a)$ for some initial policy π.

Greedily updating the policy w.r.t. $\hat{A}^{\pi}(s,a)$ can lead to a decrease (instead of an increase) in performance.

Instead, consider the more conservative policy improvement:

$$\pi_{new}(a \mid s) = (1-\alpha)\pi(a \mid s) + \alpha\pi'(a \mid s)$$

Where $\pi'(a \mid s)$ is greedy w.r.t. $\hat{A}^{\pi}(s,a)$, and $\alpha \in [0,1]$ is a *policy mixing* parameter.

Theorem 4.1: Let $J_{\rho}(\pi) = \mathbb{E}_{s\sim\rho}\left[V^{\pi}(s)\right]$ be the expected value of policy π from initial distribution ρ, and $\pi_{new}(a \mid s)$ be defined as a mixture of π and π' with mixing parameter $\alpha \in [0,1]$, then

$$J_{\rho}(\pi_{new}) - J_{\rho}(\pi) \geq \frac{\alpha \mathbb{A}^{\pi,\rho}(\pi')}{1-\gamma} - \frac{2\alpha^2\gamma \max_s \left|\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^{\pi}(s,a)\right]\right|}{(1-\gamma)(1-\gamma(1-\alpha))}$$

Theorem 4.1, Kakade & Langford 2002

Theorem 4.1: Let $J_\rho(\pi) = \mathbb{E}_{s\sim\rho}\left[V^\pi(s)\right]$ be the expected value of policy π from initial distribution ρ, and $\pi_{new}(a \mid s)$ be defined as a mixture of π and π' with mixing parameter $\alpha \in [0,1]$, then

$$J_\rho(\pi_{new}) - J_\rho(\pi) \geq \frac{\alpha \mathbb{A}^{\pi,\rho}(\pi')}{1-\gamma} - \frac{2\alpha^2\gamma \max_s \left|\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^\pi(s,a)\right]\right|}{(1-\gamma)(1-\gamma(1-\alpha))}$$

Theorem 4.1, Kakade & Langford 2002

Theorem 4.1: Let $J_\rho(\pi) = \mathbb{E}_{s\sim\rho}\left[V^\pi(s)\right]$ be the expected value of policy π from initial distribution ρ, and $\pi_{new}(a \mid s)$ be defined as a mixture of π and π' with mixing parameter $\alpha \in [0,1]$, then

$$J_\rho(\pi_{new}) - J_\rho(\pi) \geq \frac{\alpha \mathbb{A}^{\pi,\rho}(\pi')}{1-\gamma} - \frac{2\alpha^2\gamma \max_s \left|\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^\pi(s,a)\right]\right|}{(1-\gamma)(1-\gamma(1-\alpha))}$$

Difference in policy performance

Theorem 4.1, Kakade & Langford 2002

Theorem 4.1: Let $J_\rho(\pi) = \mathbb{E}_{s\sim\rho}\left[V^\pi(s)\right]$ be the expected value of policy π from initial distribution ρ, and $\pi_{new}(a \mid s)$ be defined as a mixture of π and π' with mixing parameter $\alpha \in [0,1]$, then

$$J_\rho(\pi_{new}) - J_\rho(\pi) \geq \frac{\alpha \mathbb{A}^{\pi,\rho}(\pi')}{1-\gamma} - \frac{2\alpha^2\gamma \max_s \left|\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^\pi(s,a)\right]\right|}{(1-\gamma)(1-\gamma(1-\alpha))}$$

Difference in policy performance

Normalized advantage of π' over π.

Theorem 4.1, Kakade & Langford 2002

Theorem 4.1: Let $J_\rho(\pi) = \mathbb{E}_{s\sim\rho}\left[V^\pi(s)\right]$ be the expected value of policy π from initial distribution ρ, and $\pi_{new}(a \mid s)$ be defined as a mixture of π and π' with mixing parameter $\alpha \in [0,1]$, then

$$J_\rho(\pi_{new}) - J_\rho(\pi) \geq \frac{\alpha \mathbb{A}^{\pi,\rho}(\pi')}{1-\gamma} - \frac{2\alpha^2\gamma \max_s \left|\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^\pi(s,a)\right]\right|}{(1-\gamma)(1-\gamma(1-\alpha))}$$

Difference in policy performance

Normalized advantage of π' over π.

Penalty due to π' greedily selecting actions using only approximations of the state-action values.

Theorem 4.1, Kakade & Langford 2002

Theorem 4.1: Let $J_\rho(\pi) = \mathbb{E}_{s\sim\rho}\left[V^\pi(s)\right]$ be the expected value of policy π from initial distribution ρ, and $\pi_{new}(a \mid s)$ be defined as a mixture of π and π' with mixing parameter $\alpha \in [0,1]$, then

$$J_\rho(\pi_{new}) - J_\rho(\pi) \geq \frac{\alpha \mathbb{A}^{\pi,\rho}(\pi')}{1-\gamma} - \frac{2\alpha^2\gamma \max_s \left|\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^\pi(s,a)\right]\right|}{(1-\gamma)(1-\gamma(1-\alpha))}$$

Difference in policy performance

Normalized advantage of π' over π.

Penalty due to π' greedily selecting actions using only approximations of the state-action values.

Case $\alpha = 0$:

$\pi_{new} = \pi$ and $J_\rho(\pi_{new}) - J_\rho(\pi) \geq 0$

Case $\alpha = 1$:

$$\pi_{new} = \pi' \text{ and } J_\rho(\pi_{new}) - J_\rho(\pi) \geq \frac{\mathbb{A}^{\pi,\rho}}{1-\gamma} - \frac{2\gamma\epsilon}{1-\gamma}$$

Theorem 4.1, Kakade & Langford 2002

Theorem 4.1: Let $J_\rho(\pi) = \mathbb{E}_{s\sim\rho}\left[V^\pi(s)\right]$ be the expected value of policy π from initial distribution ρ, and $\pi_{new}(a \mid s)$ be defined as a mixture of π and π' with mixing parameter $\alpha \in [0,1]$, then

$$J_\rho(\pi_{new}) - J_\rho(\pi) \geq \frac{\alpha \mathbb{A}^{\pi,\rho}(\pi')}{1-\gamma} - \frac{2\alpha^2\gamma \max_s \left|\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[A^\pi(s,a)\right]\right|}{(1-\gamma)(1-\gamma(1-\alpha))}$$

Difference in policy performance

Normalized advantage of π' over π.

Penalty due to π' greedily selecting actions using only approximations of the state-action values.

Case $\alpha = 0$:

$\pi_{new} = \pi$ and $J_\rho(\pi_{new}) - J_\rho(\pi) \geq 0$

Case $\alpha = 1$:

$$\pi_{new} = \pi' \text{ and } J_\rho(\pi_{new}) - J_\rho(\pi) \geq \frac{\mathbb{A}^{\pi,\rho}}{1-\gamma} - \frac{2\gamma\epsilon}{1-\gamma}$$

Intuition: For sufficiently small α, policy improvement is guaranteed to occur so long as the advantage is positive. For large values of α, significant degredation could occur.

Conservative policy iteration

Assume acces to an ϵ-greedy policy chooser $G_\epsilon(\pi, \rho)$, which returns a policy π' such that $\mathbb{A}^{\pi,\rho}(\pi') \geq \text{OPT}(\mathbb{A}^{\pi,\rho}(\pi')) - \epsilon$, where $\text{OPT}(\mathbb{A}^{\pi,\rho}(\pi')) = \max_{\pi'} \mathbb{A}^{\pi,\rho}(\pi')$.

5 Conservative Policy Iteration

For simplicity, we assume knowledge of ε. The conservative policy iteration algorithm is:

(1) Call $G_\varepsilon(\pi, \mu)$ to obtain some π'
(2) Use $O(\frac{R^2}{\varepsilon^2} \log \frac{R^2}{\delta\varepsilon^2})$ μ-restarts to obtain an $\frac{\varepsilon}{3}$-accurate estimate $\hat{\mathbb{A}}$ of $\mathbb{A}_{\pi,\mu}(\pi')$.
(3) If $\hat{\mathbb{A}} < \frac{2\varepsilon}{3}$, STOP and return π.
(4) If $\hat{\mathbb{A}} \geq \frac{2\varepsilon}{3}$, then update policy π according to equation 4.1 using $\frac{(1-\gamma)(\hat{\mathbb{A}}-\frac{\varepsilon}{3})}{4R}$ and return to step 1.

Theorem 4.4. *With probability at least $1-\delta$, conservative policy iteration: i) improves η_μ with every policy update, ii) ceases in at most $72\frac{R^2}{\varepsilon^2}$ calls to $G_\varepsilon(\pi, \mu)$, and iii) returns a policy π such that $OPT(\mathbb{A}_{\pi,\mu}) < 2\varepsilon$.*

Conservative policy iteration

Intuition: For sufficiently small α, policy improvement is guaranteed to occur so long as the advantage is positive.
For large values of α, significant degredation could occur.

Assume acces to an ϵ-greedy policy chooser $G_\epsilon(\pi, \rho)$, which returns a policy π' such that $\mathbb{A}^{\pi,\rho}(\pi') \geq \text{OPT}(\mathbb{A}^{\pi,\rho}(\pi')) - \epsilon$, where $\text{OPT}(\mathbb{A}^{\pi,\rho}(\pi')) = \max_{\pi'} \mathbb{A}^{\pi,\rho}(\pi')$.

5 Conservative Policy Iteration

For simplicity, we assume knowledge of ε. The conservative policy iteration algorithm is:

(1) Call $G_\varepsilon(\pi, \mu)$ to obtain some π'
(2) Use $O(\frac{R^2}{\varepsilon^2} \log \frac{R^2}{\delta\varepsilon^2})$ μ-restarts to obtain an $\frac{\varepsilon}{3}$-accurate estimate $\hat{\mathbb{A}}$ of $\mathbb{A}_{\pi,\mu}(\pi')$.
(3) If $\hat{\mathbb{A}} < \frac{2\varepsilon}{3}$, STOP and return π.
(4) If $\hat{\mathbb{A}} \geq \frac{2\varepsilon}{3}$, then update policy π according to equation 4.1 using $\frac{(1-\gamma)(\hat{\mathbb{A}}-\frac{\varepsilon}{3})}{4R}$ and return to step 1.

Theorem 4.4. *With probability at least $1-\delta$, conservative policy iteration: i) improves η_μ with every policy update, ii) ceases in at most $72\frac{R^2}{\varepsilon^2}$ calls to $G_\varepsilon(\pi, \mu)$, and iii) returns a policy π such that $OPT(\mathbb{A}_{\pi,\mu}) < 2\varepsilon$.*

Conservative policy iteration

Intuition: For sufficiently small α, policy improvement is guaranteed to occur so long as the advantage is positive. For large values of α, significant degredation could occur.

Corollary 4.2. *Let R be the maximal possible reward and $\mathbb{A}$ be the policy advantage of π' with respect to π and μ. If $\mathbb{A} \geq 0$, then using $\alpha = \frac{(1-\gamma)\mathbb{A}}{4R}$ guarantees the following policy improvement:*

$$\eta_\mu(\pi_{new}) - \eta_\mu(\pi) \geq \frac{\mathbb{A}^2}{8R}.$$

Assume acces to an ϵ-greedy policy chooser $G_\epsilon(\pi, \rho)$, which returns a policy π' such that $\mathbb{A}^{\pi,\rho}(\pi') \geq \text{OPT}(\mathbb{A}^{\pi,\rho}(\pi')) - \epsilon$, where $\text{OPT}(\mathbb{A}^{\pi,\rho}(\pi')) = \max_{\pi'} \mathbb{A}^{\pi,\rho}(\pi')$.

5 Conservative Policy Iteration

For simplicity, we assume knowledge of ε. The conservative policy iteration algorithm is:

(1) Call $G_\varepsilon(\pi, \mu)$ to obtain some π'
(2) Use $O(\frac{R^2}{\varepsilon^2} \log \frac{R^2}{\delta\varepsilon^2})$ μ-restarts to obtain an $\frac{\varepsilon}{3}$-accurate estimate $\hat{\mathbb{A}}$ of $\mathbb{A}_{\pi,\mu}(\pi')$.
(3) If $\hat{\mathbb{A}} < \frac{2\varepsilon}{3}$, STOP and return π.
(4) If $\hat{\mathbb{A}} \geq \frac{2\varepsilon}{3}$, then update policy π according to equation 4.1 using $\frac{(1-\gamma)(\hat{\mathbb{A}}-\frac{\varepsilon}{3})}{4R}$ and return to step 1.

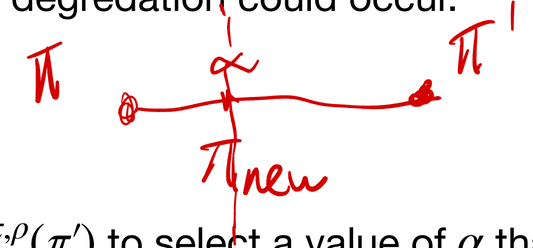
Theorem 4.4. *With probability at least $1-\delta$, conservative policy iteration: i) improves η_μ with every policy update, ii) ceases in at most $72\frac{R^2}{\varepsilon^2}$ calls to $G_\varepsilon(\pi, \mu)$, and iii) returns a policy π such that $OPT(\mathbb{A}_{\pi,\mu}) < 2\varepsilon$.*

Conservative policy iteration

Intuition: For sufficiently small α, policy improvement is guaranteed to occur so long as the advantage is positive. For large values of α, significant degredation could occur.

Corollary 4.2. *Let R be the maximal possible reward and $\mathbb{A}$ be the policy advantage of π' with respect to π and μ. If $\mathbb{A} \geq 0$, then using $\alpha = \frac{(1-\gamma)\mathbb{A}}{4R}$ guarantees the following policy improvement:*

$$\eta_\mu(\pi_{new}) - \eta_\mu(\pi) \geq \frac{\mathbb{A}^2}{8R}.$$



CPI idea: Use an estimate of $\hat{\mathbb{A}}^{\pi,\rho}(\pi')$ to select a value of α that guarantees policy improvement at each step.

Assume acces to an ϵ-greedy policy chooser $G_\epsilon(\pi, \rho)$, which returns a policy π' such that $\mathbb{A}^{\pi,\rho}(\pi') \geq \text{OPT}(\mathbb{A}^{\pi,\rho}(\pi')) - \epsilon$, where $\text{OPT}(\mathbb{A}^{\pi,\rho}(\pi')) = \max_{\pi'} \mathbb{A}^{\pi,\rho}(\pi')$.

5 Conservative Policy Iteration

For simplicity, we assume knowledge of ε. The conservative policy iteration algorithm is:

(1) Call $G_\varepsilon(\pi, \mu)$ to obtain some π'
(2) Use $O(\frac{R^2}{\varepsilon^2} \log \frac{R^2}{\delta\varepsilon^2})$ μ-restarts to obtain an $\frac{\varepsilon}{3}$-accurate estimate $\hat{\mathbb{A}}$ of $\mathbb{A}_{\pi,\mu}(\pi')$.
(3) If $\hat{\mathbb{A}} < \frac{2\varepsilon}{3}$, STOP and return π.
(4) If $\hat{\mathbb{A}} \geq \frac{2\varepsilon}{3}$, then update policy π according to equation 4.1 using $\frac{(1-\gamma)(\hat{\mathbb{A}}-\frac{\varepsilon}{3})}{4R}$ and return to step 1.

Theorem 4.4. *With probability at least $1-\delta$, conservative policy iteration: i) improves η_μ with every policy update, ii) ceases in at most $72\frac{R^2}{\varepsilon^2}$ calls to $G_\varepsilon(\pi, \mu)$, and iii) returns a policy π such that $OPT(\mathbb{A}_{\pi,\mu}) < 2\varepsilon$.*

Core ideas of the CPI paper

- A bound on performance of policy improvement steps that rely on approximate state-action values.

Key idea: Important to take sufficiently small steps when improving our policy.

- Since the paper assumes access to ϵ-greedy policy picker, it's not quite implementable.

How can we address these issues?

TRPO

Trust region policy optimization

Trust Region Policy Optimization

John Schulman JOSCHU@EECS.BERKELEY.EDU
Sergey Levine SLEVINE@EECS.BERKELEY.EDU
Philipp Moritz PCMORITZ@EECS.BERKELEY.EDU
Michael Jordan JORDAN@CS.BERKELEY.EDU
Pieter Abbeel PABBEEL@CS.BERKELEY.EDU
University of California, Berkeley, Department of Electrical Engineering and Computer Sciences

Abstract

We describe an iterative procedure for optimizing policies, with guaranteed monotonic improvement. By making several approximations to the theoretically-justified procedure, we develop a practical algorithm, called Trust Region Policy Optimization (TRPO). This algorithm is similar to natural policy gradient methods and is effective for optimizing large nonlinear policies such as neural networks. Our experiments demonstrate its robust performance on a wide variety of tasks: learning simulated robotic swimming, hopping, and walking gaits; and playing Atari games using images of the screen as input. Despite its approximations that deviate from the theory, TRPO tends to give monotonic improvement, with little tuning of hyperparameters.

1 Introduction

Most algorithms for policy optimization can be classified into three broad categories: (1) policy iteration methods, which alternate between estimating the value function under the current policy and improving the policy (Bertsekas, 2005); (2) policy gradient methods, which use an estimator of the gradient of the expected return (total reward) ob-

Tetris is a classic benchmark problem for approximate dynamic programming (ADP) methods, stochastic optimization methods are difficult to beat on this task (Gabillon et al., 2013). For continuous control problems, methods like CMA have been successful at learning control policies for challenging tasks like locomotion when provided with hand-engineered policy classes with low-dimensional parameterizations (Wampler & Popović, 2009). The inability of ADP and gradient-based methods to consistently beat gradient-free random search is unsatisfying, since gradient-based optimization algorithms enjoy much better sample complexity guarantees than gradient-free methods (Nemirovski, 2005). Continuous gradient-based optimization has been very successful at learning function approximators for supervised learning tasks with huge numbers of parameters, and extending their success to reinforcement learning would allow for efficient training of complex and powerful policies.

In this article, we first prove that minimizing a certain surrogate objective function guarantees policy improvement with non-trivial step sizes. Then we make a series of approximations to the theoretically-justified algorithm, yielding a practical algorithm, which we call trust region policy optimization (TRPO). We describe two variants of this algorithm: first, the *single-path* method, which can be applied in the model-free setting; second, the *vine* method,

1. A generalization of Kakade & Langford's theoretical results,
2. A theoretically-justified policy optimization algorithm (with guaranteed monotic improvement).
3. A practical algorithm (TRPO) inspired by the theory, that makes several simplifications and that works well in practice.

A generalization of Kakade & Langford's theoretical results

Kakade & Langford, Lemma 6.1: For any policies $\tilde{\pi}$ and π and any starting state distribution ρ

$$J_\rho(\tilde{\pi}) = J_\rho(\pi) + \frac{1}{1-\gamma}\mathbb{E}_{a_0,s_0,\ldots\sim\tilde{\pi}}\left[A^\pi(s,a)\right]$$

A generalization of Kakade & Langford's theoretical results

Kakade & Langford, Lemma 6.1: For any policies $\tilde{\pi}$ and π and any starting state distribution ρ

$$J_\rho(\tilde{\pi}) = J_\rho(\pi) + \frac{1}{1-\gamma}\mathbb{E}_{a_0,s_0,\ldots\sim\tilde{\pi}}\left[A^\pi(s,a)\right]$$

Let $\rho_\pi(s) = P(s_0 = s) + \gamma P(s_1 = s) + \gamma^2 P(s_2 = s) + \ldots$ be the unnormalized discounted visitation frequencies, with $s_0 \sim \rho$.

A generalization of Kakade & Langford's theoretical results

Kakade & Langford, Lemma 6.1: For any policies $\tilde{\pi}$ and π and any starting state distribution ρ

$$J_\rho(\tilde{\pi}) = J_\rho(\pi) + \frac{1}{1-\gamma}\mathbb{E}_{a_0,s_0,\ldots\sim\tilde{\pi}}\left[A^\pi(s,a)\right]$$

Let $\rho_\pi(s) = P(s_0 = s) + \gamma P(s_1 = s) + \gamma^2 P(s_2 = s) + \ldots$ be the unnormalized discounted visitation frequencies, with $s_0 \sim \rho$.

Then we can re-write Lemma 6.1 as: $J_\rho(\tilde{\pi}) = J_\rho(\pi) + \sum_s \rho_{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a \mid s) A^\pi(s,a)$.

Note, can't optimize this because we can't approximate $\rho_{\tilde{\pi}}(s)$, much easier to get $\rho_\pi(s)$ for the current policy π.

A generalization of Kakade & Langford's theoretical results

Kakade & Langford, Lemma 6.1: For any policies $\tilde{\pi}$ and π and any starting state distribution ρ

$$J_\rho(\tilde{\pi}) = J_\rho(\pi) + \frac{1}{1-\gamma}\mathbb{E}_{a_0,s_0,\ldots\sim\tilde{\pi}}\left[A^\pi(s,a)\right]$$

Let $\rho_\pi(s) = P(s_0 = s) + \gamma P(s_1 = s) + \gamma^2 P(s_2 = s) + \ldots$ be the unnormalized discounted visitation frequencies, with $s_0 \sim \rho$.

Then we can re-write Lemma 6.1 as: $J_\rho(\tilde{\pi}) = J_\rho(\pi) + \sum_s \rho_{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a \mid s) A^\pi(s,a)$.

Note, can't optimize this because we can't approximate $\rho_{\tilde{\pi}}(s)$, much easier to get $\rho_\pi(s)$ for the current policy π.

Instead, define a *local* approximation: $L_\pi(\tilde{\pi}) = J_\rho(\pi) + \sum_s \rho_\pi(s) \sum_a \tilde{\pi}(a \mid s) A^\pi(s,a)$ Uses $\rho_\pi(s)$ instead of $\rho_{\tilde{\pi}}(s)$.

Note, this is a local approximation in the sense that $L_{\pi_{\theta_0}}(\pi_{\theta_0}) = J_\rho(\pi_{\theta_0})$, and $\nabla_\theta L_{\pi_{\theta_0}}(\pi_\theta)\big|_{\theta=\theta_0} = \nabla_\theta J_\rho(\pi_\theta)\big|_{\theta=\theta_0}$.

But, as π_θ moves further away from π_{θ_0}, this approximation stops working.

A generalization of Kakade & Langford's theoretical results

By re-writing Kakade & Langford's Theorem 4.1 in terms of $L_\pi(\tilde{\pi})$ we can gain more insight into the result.

$$J_\rho(\pi_{new}) - J_\rho(\pi) \geq \frac{\alpha \mathbb{A}^{\pi,\rho}(\pi')}{1-\gamma} - \frac{2\alpha^2\gamma \max_s \left| \mathbb{E}_{a\sim\pi'(\cdot|s)} \left[A^\pi(s,a) \right] \right|}{(1-\gamma)(1-\gamma(1-\alpha))} \implies J_\rho(\pi_{new}) \geq L_\pi(\pi_{new}) - \frac{2\gamma \max_s \left| \mathbb{E}_{a\sim\pi'(a|s)} \left[A^\pi(s,a) \right] \right|}{(1-\gamma)^2}\alpha^2$$

A generalization of Kakade & Langford's theoretical results

Instead, define a *local* approximation: $L_\pi(\tilde{\pi}) = J_\rho(\pi) + \sum_s \rho_\pi(s) \sum_a \tilde{\pi}(a \mid s) A^\pi(s,a)$ Uses $\rho_\pi(s)$ instead of $\rho_{\tilde{\pi}}(s)$.

Note, this is a local approximation in the sense that $L_{\pi_{\theta_0}}(\pi_{\theta_0}) = J_\rho(\pi_{\theta_0})$, and $\nabla_\theta L_{\pi_{\theta_0}}(\pi_\theta)|_{\theta=\theta_0} = \nabla_\theta J_\rho(\pi_\theta)|_{\theta=\theta_0}$.

But, as π_θ moves further away from π_{θ_0}, this approximation stops working.

By re-writing Kakade & Langford's Theorem 4.1 in terms of $L_\pi(\tilde{\pi})$ we can gain more insight into the result.

$$J_\rho(\pi_{new}) - J_\rho(\pi) \geq \frac{\alpha \mathbb{A}^{\pi,\rho}(\pi')}{1-\gamma} - \frac{2\alpha^2 \gamma \max_s \left| \mathbb{E}_{a\sim\pi'(\cdot|s)} \left[A^\pi(s,a) \right] \right|}{(1-\gamma)(1-\gamma(1-\alpha))} \implies J_\rho(\pi_{new}) \geq L_\pi(\pi_{new}) - \frac{2\gamma \max_s \left| \mathbb{E}_{a\sim\pi'(a|s)} \left[A^\pi(s,a) \right] \right|}{(1-\gamma)^2} \alpha^2$$

- Remember π_{new} was written as a mixture of π and π'. In practice, people don't implement policy mixtures like this.
- How can we define some other notion of "magnitude" of policy update? Total variation divergence!

A generalization of Kakade & Langford's theoretical results

Idea: use the total variation divergence, instead of parameterized policy mixing, as the notion of how "big" a particular policy improvement step is.

For discrete distributions p and q, $D_{TV}(p \,||\, q) = \frac{1}{2} \sum_i \left| p_i - q_i \right|$.

Define $D_{TV}^{max}(\pi, \tilde{\pi}) = \max_s D_{TV}\left(\pi(\,\cdot \mid s) \,||\, \tilde{\pi}(\,\cdot \mid s)\right)$.

A generalization of Kakade & Langford's theoretical results

Idea: use the total variation divergence, instead of parameterized policy mixing, as the notion of how "big" a particular policy improvement step is.

For discrete distributions p and q, $D_{TV}(p\,||\,q) = \frac{1}{2}\sum_i \left|p_i - q_i\right|$.

Define $D_{TV}^{max}(\pi, \tilde{\pi}) = \max_s D_{TV}\left(\pi(\cdot \mid s)\,||\,\tilde{\pi}(\cdot \mid s)\right)$.

Schulman et al. Theorem 1: Let $\alpha = D_{TV}^{max}(\pi_{old}, \pi_{new})$, then:

$$J_\rho(\pi_{new}) \geq L_{\pi_{old}}(\pi_{new}) - \frac{4\gamma \max_{s,a}\left|A^\pi(s,a)\right|}{(1-\gamma)^2}\alpha^2$$

A generalization of Kakade & Langford's theoretical results

Idea: use the total variation divergence, instead of parameterized policy mixing, as the notion of how "big" a particular policy improvement step is.

For discrete distributions p and q, $D_{TV}(p||q) = \frac{1}{2}\sum_i \left|p_i - q_i\right|$.

Define $D_{TV}^{max}(\pi, \tilde{\pi}) = \max_s D_{TV}\left(\pi(\cdot \mid s)||\tilde{\pi}(\cdot \mid s)\right)$.

Schulman et al. Theorem 1: Let $\alpha = D_{TV}^{max}(\pi_{old}, \pi_{new})$, then:

$$J_\rho(\pi_{new}) \geq L_{\pi_{old}}(\pi_{new}) - \frac{4\gamma \max_{s,a}\left|A^\pi(s,a)\right|}{(1-\gamma)^2}\alpha^2$$

Using the fact that $D_{KL}(p||q) \geq D_{TV}(p||q)^2$, we immediately arrive at the corrolary:

$$J_\rho(\pi_{new}) \geq L_{\pi_{old}}(\pi_{new}) - CD_{KL}^{max}(\pi_{old}, \pi_{new}) \quad \text{Where } C = \frac{4\gamma \max_{s,a}\left|A^\pi(s,a)\right|}{(1-\gamma)^2}$$

How can we use this in an algorithm?

A theoretically-justified policy optimization algorithm

Idea: maximize the lower bound provided by the theorem.

Algorithm 1 Policy iteration algorithm guaranteeing non-decreasing expected return η

Initialize π_0.

for $i = 0, 1, 2, \ldots$ until convergence **do**

Compute all advantage values $A_{\pi_i}(s, a)$.

Solve the constrained optimization problem

$$\pi_{i+1} = \arg\max_{\pi} \left[L_{\pi_i}(\pi) - C D_{\mathrm{KL}}^{\max}(\pi_i, \pi) \right]$$

where $C = 4\epsilon\gamma/(1-\gamma)^2$

and $L_{\pi_i}(\pi) = \eta(\pi_i) + \sum_s \rho_{\pi_i}(s) \sum_a \pi(a|s) A_{\pi_i}(s, a)$

end for

A theoretically-justified policy optimization algorithm

Idea: maximize the lower bound provided by the theorem.

Algorithm 1 Policy iteration algorithm guaranteeing non-decreasing expected return η

Initialize π_0.

for $i = 0, 1, 2, \ldots$ until convergence **do**

Compute all advantage values $A_{\pi_i}(s, a)$.

Solve the constrained optimization problem

$$\pi_{i+1} = \arg\max_{\pi} \left[L_{\pi_i}(\pi) - C D_{\mathrm{KL}}^{\max}(\pi_i, \pi) \right]$$

where $C = 4\epsilon\gamma/(1-\gamma)^2$

and $L_{\pi_i}(\pi) = \eta(\pi_i) + \sum_s \rho_{\pi_i}(s) \sum_a \pi(a|s) A_{\pi_i}(s, a)$

end for

This algorithm is guaranteed to produce a monotonically improving sequence of policies $J(\pi_0) \leq J(\pi_1) \leq J(\pi_2) \leq \ldots$

A theoretically-justified policy optimization algorithm

Idea: maximize the lower bound provided by the theorem.

Algorithm 1 Policy iteration algorithm guaranteeing non-decreasing expected return η

Initialize π_0.

for $i = 0, 1, 2, \ldots$ until convergence **do**

Compute all advantage values $A_{\pi_i}(s, a)$.

Solve the constrained optimization problem

$$\pi_{i+1} = \arg\max_{\pi} \left[L_{\pi_i}(\pi) - C D_{\mathrm{KL}}^{\max}(\pi_i, \pi) \right]$$

where $C = 4\epsilon\gamma/(1-\gamma)^2$

and $L_{\pi_i}(\pi) = \eta(\pi_i) + \sum_s \rho_{\pi_i}(s) \sum_a \pi(a|s) A_{\pi_i}(s, a)$

end for

This algorithm is guaranteed to produce a monotonically improving sequence of policies $J(\pi_0) \leq J(\pi_1) \leq J(\pi_2) \leq \ldots$

Let $M_i(\pi) = L_{\pi_i}(\pi) - C D_{TV}^{max}(\pi_i, \pi)$.

A theoretically-justified policy optimization algorithm

Idea: maximize the lower bound provided by the theorem.

Algorithm 1 Policy iteration algorithm guaranteeing non-decreasing expected return η

Initialize π_0.

for $i = 0, 1, 2, \ldots$ until convergence **do**

Compute all advantage values $A_{\pi_i}(s, a)$.

Solve the constrained optimization problem

$$\pi_{i+1} = \arg\max_{\pi} \left[L_{\pi_i}(\pi) - C D_{\mathrm{KL}}^{\max}(\pi_i, \pi) \right]$$

where $C = 4\epsilon\gamma/(1-\gamma)^2$

and $L_{\pi_i}(\pi) = \eta(\pi_i) + \sum_s \rho_{\pi_i}(s) \sum_a \pi(a|s) A_{\pi_i}(s, a)$

end for

This algorithm is guaranteed to produce a monotonically improving sequence of policies $J(\pi_0) \leq J(\pi_1) \leq J(\pi_2) \leq \ldots$

Let $M_i(\pi) = L_{\pi_i}(\pi) - C D_{TV}^{max}(\pi_i, \pi)$.

Then, $J(\pi_{i+1}) \geq M_i(\pi_{i+1})$ by the corollary.

A theoretically-justified policy optimization algorithm

Idea: maximize the lower bound provided by the theorem.

Algorithm 1 Policy iteration algorithm guaranteeing non-decreasing expected return η

Initialize π_0.

for $i = 0, 1, 2, \ldots$ until convergence **do**

Compute all advantage values $A_{\pi_i}(s, a)$.

Solve the constrained optimization problem

$$\pi_{i+1} = \arg\max_{\pi} \left[L_{\pi_i}(\pi) - C D_{\mathrm{KL}}^{\max}(\pi_i, \pi) \right]$$

where $C = 4\epsilon\gamma/(1-\gamma)^2$

and $L_{\pi_i}(\pi) = \eta(\pi_i) + \sum_s \rho_{\pi_i}(s) \sum_a \pi(a|s) A_{\pi_i}(s, a)$

end for

This algorithm is guaranteed to produce a monotonically improving sequence of policies $J(\pi_0) \leq J(\pi_1) \leq J(\pi_2) \leq \ldots$

Let $M_i(\pi) = L_{\pi_i}(\pi) - C D_{TV}^{max}(\pi_i, \pi)$.

Then, $J(\pi_{i+1}) \geq M_i(\pi_{i+1})$ by the corollary.

And, $M_i(\pi_i) = L_{\pi_i}(\pi_i) - C D_{TV}^{max}(\pi_i, \pi_i) = J(\pi_i)$.

A theoretically-justified policy optimization algorithm

Idea: maximize the lower bound provided by the theorem.

Algorithm 1 Policy iteration algorithm guaranteeing non-decreasing expected return η

Initialize π_0.

for $i = 0, 1, 2, \ldots$ until convergence **do**

Compute all advantage values $A_{\pi_i}(s, a)$.

Solve the constrained optimization problem

$$\pi_{i+1} = \arg\max_{\pi} \left[L_{\pi_i}(\pi) - C D_{\mathrm{KL}}^{\max}(\pi_i, \pi) \right]$$

where $C = 4\epsilon\gamma/(1-\gamma)^2$

and $L_{\pi_i}(\pi) = \eta(\pi_i) + \sum_s \rho_{\pi_i}(s) \sum_a \pi(a|s) A_{\pi_i}(s, a)$

end for

This algorithm is guaranteed to produce a monotonically improving sequence of policies $J(\pi_0) \leq J(\pi_1) \leq J(\pi_2) \leq \ldots$

Let $M_i(\pi) = L_{\pi_i}(\pi) - C D_{TV}^{max}(\pi_i, \pi)$.

Then, $J(\pi_{i+1}) \geq M_i(\pi_{i+1})$ by the corollary.

And, $M_i(\pi_i) = L_{\pi_i}(\pi_i) - C D_{TV}^{max}(\pi_i, \pi_i) = J(\pi_i)$.

Therefore, $J(\pi_{i+1}) - J(\pi_i) \geq M_i(\pi_{i+1}) - M_i(\pi_i)$.

A theoretically-justified policy optimization algorithm

Idea: maximize the lower bound provided by the theorem.

Algorithm 1 Policy iteration algorithm guaranteeing non-decreasing expected return η

Initialize π_0.

for $i = 0, 1, 2, \ldots$ until convergence **do**

Compute all advantage values $A_{\pi_i}(s, a)$.

Solve the constrained optimization problem

$$\pi_{i+1} = \arg\max_{\pi} \left[L_{\pi_i}(\pi) - C D_{\mathrm{KL}}^{\max}(\pi_i, \pi) \right]$$

where $C = 4\epsilon\gamma/(1-\gamma)^2$

and $L_{\pi_i}(\pi) = \eta(\pi_i) + \sum_s \rho_{\pi_i}(s) \sum_a \pi(a|s) A_{\pi_i}(s, a)$

end for

This algorithm is guaranteed to produce a monotonically improving sequence of policies $J(\pi_0) \leq J(\pi_1) \leq J(\pi_2) \leq \ldots$

Let $M_i(\pi) = L_{\pi_i}(\pi) - C D_{TV}^{max}(\pi_i, \pi)$.

Then, $J(\pi_{i+1}) \geq M_i(\pi_{i+1})$ by the corollary.

And, $M_i(\pi_i) = L_{\pi_i}(\pi_i) - C D_{TV}^{max}(\pi_i, \pi_i) = J(\pi_i)$.

Therefore, $J(\pi_{i+1}) - J(\pi_i) \geq M_i(\pi_{i+1}) - M_i(\pi_i)$.

So, maximizing $M_i(\pi)$ at each iteration guarantees the true objective $J(\pi_i)$ is non-decreasing.

However, this algorithm is not practical
↳ it's expensive to compute C
and $D_{KL}^{max}(\pi_i, \pi)$ requires a maximization problem over the state space.

The trust region policy optimization (TRPO) algorithm

Objective for policy improvement step: $\max_{\theta} \left[L_{\theta_{old}}(\theta) - C D_{TV}^{max}(\theta_{old}, \theta) \right]$

where $C = \frac{4\gamma \max_{s,a} \left| A^{\pi}(s,a) \right|}{(1-\gamma)^2}$

The trust region policy optimization (TRPO) algorithm

Objective for policy improvement step: $\max_{\theta} \left[L_{\theta_{old}}(\theta) - C D_{TV}^{max}(\theta_{old}, \theta) \right]$ where $C = \frac{4\gamma \max_{s,a} \left| A^{\pi}(s,a) \right|}{(1-\gamma)^2}$

Challenge 1: the theoretically-justified C results in very small step sizes.
Also, it is challenging to compute C accurately, and difficult to pick a constant algorithm hyperparameter to use as C across different tasks/environments.

Solution to challenge 1: Don't solve $\max_{\theta} \left[L_{\theta_{old}}(\theta) - C D_{TV}^{max}(\theta_{old}, \theta) \right]$, instead solve the constrained maximization problem:

$$\max_{\theta} L_{\theta_{old}}(\theta)$$
$$\text{s.t. } D_{KL}^{max}(\theta_{old}, \theta) \leq \delta$$

The trust region policy optimization (TRPO) algorithm

Objective for policy improvement step: $\max_{\theta} \left[L_{\theta_{old}}(\theta) - C D_{TV}^{max}(\theta_{old}, \theta) \right]$ where $C = \frac{4\gamma \max_{s,a} \left| A^{\pi}(s,a) \right|}{(1-\gamma)^2}$

Challenge 1: the theoretically-justified C results in very small step sizes.
Also, it is challenging to compute C accurately, and difficult to pick a constant algorithm hyperparameter to use as C across different tasks/environments.

Solution to challenge 1: Don't solve $\max_{\theta} \left[L_{\theta_{old}}(\theta) - C D_{TV}^{max}(\theta_{old}, \theta) \right]$, instead solve the constrained maximization problem:

$$\max_{\theta} L_{\theta_{old}}(\theta)$$
$$\text{s.t. } D_{KL}^{max}(\theta_{old}, \theta) \leq \delta$$

Challenge 2: $D_{TV}^{max}(\theta_{old}, \theta)$ imposes a constraint on each state in the state space. This is hard to enforce in practice.

Solution to challenge 2: Replace $D_{TV}^{max}(\theta_{old}, \theta)$ with an *average* KL over states:

$$\max_{\theta} L_{\theta_{old}}(\theta)$$
$$\text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

where $\bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) = \mathbb{E}_{s \sim \rho_{\theta_{old}}} \left[D_{KL} \left(\pi_{\theta_{old}}(\cdot \mid s) \,||\, \pi_{\theta}(\cdot \mid s) \right) \right]$

The trust region policy optimization (TRPO) algorithm

The trust region policy optimization (TRPO) algorithm

$$\max_{\theta} L_{\theta_{old}}(\theta)$$

$$\text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

The trust region policy optimization (TRPO) algorithm

$$\max_{\theta} L_{\theta_{old}}(\theta) \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta \quad \Longrightarrow \quad \max_{\theta} \sum_{s} \rho_{\theta_{old}}(s) \sum_{a} \pi_{\theta}(a \mid s) A^{\pi_{\theta_{old}}}(s, a) \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

The trust region policy optimization (TRPO) algorithm

$$\max_{\theta} L_{\theta_{old}}(\theta) \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta \implies \max_{\theta} \sum_{s} \rho_{\theta_{old}}(s) \sum_{a} \pi_{\theta}(a \mid s) A^{\pi_{\theta_{old}}}(s, a) \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

Replace $\sum_{s} \rho_{\theta_{old}}(s)$ with an expectation over states sampled from $\rho_{\theta_{old}}$.

$$\implies \max_{\theta} \mathbb{E}_{s \sim \rho_{\theta_{old}}} \left[\sum_{a} \pi_{\theta}(a \mid s) A^{\pi_{\theta_{old}}}(s, a) \right] \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

The trust region policy optimization (TRPO) algorithm

$$\max_{\theta} L_{\theta_{old}}(\theta) \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta \quad \Longrightarrow \quad \max_{\theta} \sum_{s} \rho_{\theta_{old}}(s) \sum_{a} \pi_{\theta}(a \mid s) A^{\pi_{\theta_{old}}}(s,a) \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

Replace $\sum_{s} \rho_{\theta_{old}}(s)$ with an expectation over states sampled from $\rho_{\theta_{old}}$.

$$\Longrightarrow \quad \max_{\theta} \mathbb{E}_{s \sim \rho_{\theta_{old}}} \left[\sum_{a} \pi_{\theta}(a \mid s) A^{\pi_{\theta_{old}}}(s,a) \right] \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

Replace $\sum_{a} \pi_{\theta}(a \mid s)$ with importance sampling expectation under behavioral policy $\pi_{\theta_{old}}$.

$$\Longrightarrow \quad \max_{\theta} \mathbb{E}_{s \sim \rho_{\theta_{old}}, a \sim \pi_{\theta_{old}}(\cdot|s)} \left[\frac{\pi_{\theta}(a \mid s)}{\pi_{\theta_{old}}(a \mid s)} A^{\pi_{\theta_{old}}}(s,a) \right] \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

The trust region policy optimization (TRPO) algorithm

$$\max_{\theta} L_{\theta_{old}}(\theta) \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

$$\Longrightarrow \quad \max_{\theta} \sum_{s} \rho_{\theta_{old}}(s) \sum_{a} \pi_{\theta}(a \mid s) A^{\pi_{\theta_{old}}}(s,a) \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

Replace $\sum_{s} \rho_{\theta_{old}}(s)$ with an expectation over states sampled from $\rho_{\theta_{old}}$.

$$\Longrightarrow \quad \max_{\theta} \mathbb{E}_{s \sim \rho_{\theta_{old}}} \left[\sum_{a} \pi_{\theta}(a \mid s) A^{\pi_{\theta_{old}}}(s,a) \right] \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

Replace $\sum_{a} \pi_{\theta}(a \mid s)$ with importance sampling expectation under behavioral policy $\pi_{\theta_{old}}$.

$$\Longrightarrow \quad \max_{\theta} \mathbb{E}_{s \sim \rho_{\theta_{old}}, a \sim \pi_{\theta_{old}}(\cdot|s)} \left[\frac{\pi_{\theta}(a \mid s)}{\pi_{\theta_{old}}(a \mid s)} A^{\pi_{\theta_{old}}}(s,a) \right] \quad \text{s.t. } \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta$$

Replace advantage with Q function, and put in the definition of the average KL term.

$$\Longrightarrow \quad \max_{\theta} \mathbb{E}_{s \sim \rho_{\theta_{old}}, a \sim \pi_{\theta_{old}}(\cdot|s)} \left[\frac{\pi_{\theta}(a \mid s)}{\pi_{\theta_{old}}(a \mid s)} Q^{\pi_{\theta_{old}}}(s,a) \right] \quad \text{s.t. } \mathbb{E}_{s \sim \rho_{\theta_{old}}} \left[D_{KL}(\pi_{\theta_{old}}(\cdot \mid s) \,||\, \pi_{\theta}(\cdot \mid s)) \right] \leq \delta$$

The trust region policy optimization (TRPO) algorithm

Algorithm steps:

1. Under the current policy π_θ, collect a set of state-action pairs along with Monte Carlo estimates of their Q-values.

The trust region policy optimization (TRPO) algorithm

Algorithm steps:

1. Under the current policy π_θ, collect a set of state-action pairs along with Monte Carlo estimates of their Q-values.

2. By averaging over samples, construct the estimated objective and constraint:

$$\max_\theta \mathbb{E}_{s\sim\rho_{\theta_{old}}, a\sim\pi_{\theta_{old}}(\cdot|s)} \left[\frac{\pi_\theta(a \mid s)}{\pi_{\theta_{old}}(a \mid s)} Q^{\pi_{\theta_{old}}}(s,a) \right]$$

$$\text{s.t. } \mathbb{E}_{s\sim\rho_{\theta_{old}}} \left[D_{KL}(\pi_{\theta_{old}}(\cdot \mid s) \,||\, \pi_\theta(\cdot \mid s)) \right] \leq \delta$$

The trust region policy optimization (TRPO) algorithm

Algorithm steps:

1. Under the current policy π_θ, collect a set of state-action pairs along with Monte Carlo estimates of their Q-values.

2. By averaging over samples, construct the estimated objective and constraint:

$$\max_\theta \mathbb{E}_{s\sim\rho_{\theta_{old}},a\sim\pi_{\theta_{old}}(\cdot|s)}\left[\frac{\pi_\theta(a\mid s)}{\pi_{\theta_{old}}(a\mid s)}Q^{\pi_{\theta_{old}}}(s,a)\right]$$

$$\text{s.t. } \mathbb{E}_{s\sim\rho_{\theta_{old}}}\left[D_{KL}(\pi_{\theta_{old}}(\cdot\mid s)\,||\,\pi_\theta(\cdot\mid s))\right]\le\delta$$

3. Approximately solve the constrained optimization problem to update the policy's parameter vector. To do so, use the conjugate gradient algorithm followed by a line search.

1. Compute a search direction, using a linear approx of the objective and a quadratic approximation of the constraint.

2. Perform a line search in that direction, to improve objective while satisfying constraint.

TRPO experimental results

Tested on:

- Locomotion tasks in Mujoco (these have become a standard RL benchmark)
- Atari games, using raw images as input.

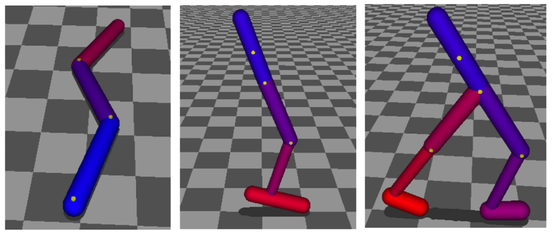


Figure 2. 2D robot models used for locomotion experiments. From left to right: swimmer, hopper, walker. The hopper and walker present a particular challenge, due to underactuation and contact discontinuities.

TRPO experimental results

Tested on:

- Locomotion tasks in Mujoco (these have become a standard RL benchmark)
- Atari games, using raw images as input.

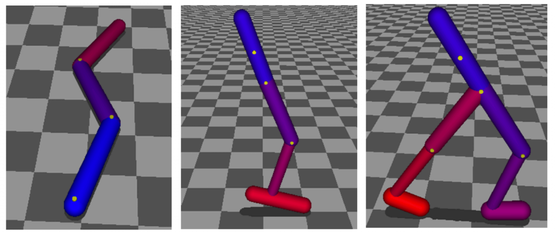


Figure 2. 2D robot models used for locomotion experiments. From left to right: swimmer, hopper, walker. The hopper and walker present a particular challenge, due to underactuation and contact discontinuities.

Locomotion results

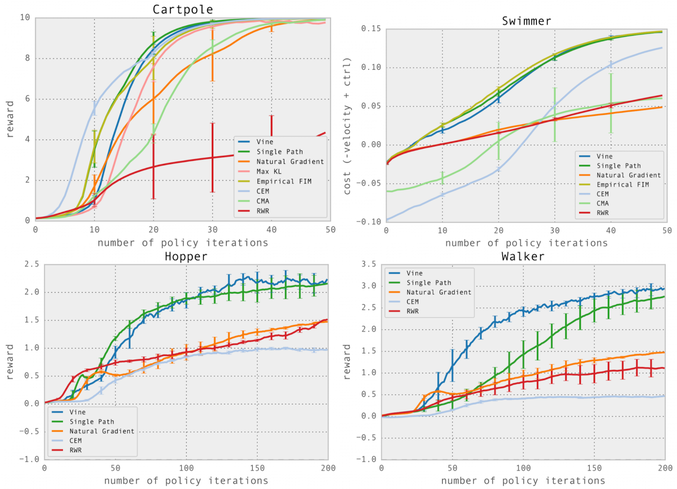


Figure 4. Learning curves for locomotion tasks, averaged across five runs of each algorithm with random initializations. Note that for the hopper and walker, a score of -1 is achievable without any forward velocity, indicating a policy that simply learned balanced standing, but not walking.

"Note that TRPO learned all of the gaits with general-purpose policies and simple reward functions, using minimal prior knowledge. This is in contrast with most prior methods for learning locomotion, which typically rely on hand-architected policy classes that explicitly encode notions of balance and stepping."

TRPO experimental results

Tested on:

- Locomotion tasks in Mujoco (these have become a standard RL benchmark)
- Atari games, using raw images as input.

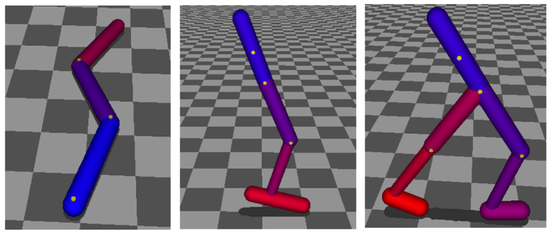


Figure 2. 2D robot models used for locomotion experiments. From left to right: swimmer, hopper, walker. The hopper and walker present a particular challenge, due to underactuation and contact discontinuities.

Locomotion results

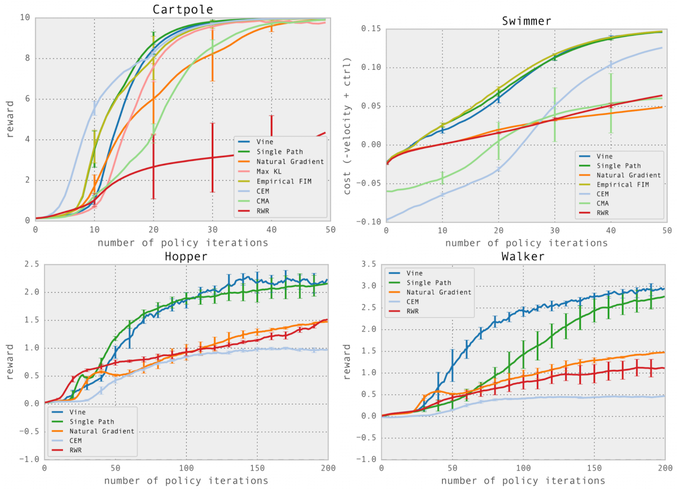


Figure 4. Learning curves for locomotion tasks, averaged across five runs of each algorithm with random initializations. Note that for the hopper and walker, a score of -1 is achievable without any forward velocity, indicating a policy that simply learned balanced standing, but not walking.

"Note that TRPO learned all of the gaits with general-purpose policies and simple reward functions, using minimal prior knowledge. This is in contrast with most prior methods for learning locomotion, which typically rely on hand-architected policy classes that explicitly encode notions of balance and stepping."

Atari results

	B. Rider	*Breakout*	*Enduro*	*Pong*	*Q*bert*	*Seaquest*	*S. Invaders*
Random	354	1.2	0	−20.4	157	110	179
Human (Mnih et al., 2013)	7456	31.0	368	−3.0	18900	28010	3690
Deep Q Learning (Mnih et al., 2013)	4092	168.0	470	20.0	1952	1705	581
UCC-I (Guo et al., 2014)	5702	380	741	21	20025	2995	692
TRPO - single path	1425.2	10.8	534.6	20.9	1973.5	1908.6	568.4
TRPO - vine	859.5	34.2	430.8	20.9	7732.5	788.4	450.2

Table 1. Performance comparison for vision-based RL algorithms on the Atari domain. Our algorithms (bottom rows) were run once on each task, with the same architecture and parameters. Performance varies substantially from run to run (with different random initializations of the policy), but we could not obtain error statistics due to time constraints.

"While our method only outperformed the prior methods on some of the games, it consistently achieved reasonable scores. … Our approach was not designed specifically for this task. The ability to apply the same policy search method to methods as diverse as robotic locomotion and image-based game playing demonstrates the generality of TRPO."

Generalized advantage estimation (GAE)

HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel
Department of Electrical Engineering and Computer Science
University of California, Berkeley
{joschu,pcmoritz,levine,jordan,pabbeel}@eecs.berkeley.edu

ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with an exponentially-weighted estimator of the advantage function that is analogous to TD(λ). We address the second challenge by using trust region optimization procedure for both the policy and the value function, which are represented by neural networks.

Our approach yields strong empirical results on highly challenging 3D locomotion tasks, learning running gaits for bipedal and quadrupedal simulated robots, and learning a policy for getting the biped to stand up from starting out lying on the ground. In contrast to a body of prior work that uses hand-crafted policy representations, our neural network policies map directly from raw kinematics to joint torques. Our algorithm is fully model-free, and the amount of simulated experi-

Used with TRPO and in PPO to balance bias-variance tradeoff in policy gradient estimates.

Generalized advantage estimation (GAE)

HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel
Department of Electrical Engineering and Computer Science
University of California, Berkeley
{joschu,pcmoritz,levine,jordan,pabbeel}@eecs.berkeley.edu

ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with an exponentially-weighted estimator of the advantage function that is analogous to TD(λ). We address the second challenge by using trust region optimization procedure for both the policy and the value function, which are represented by neural networks.

Our approach yields strong empirical results on highly challenging 3D locomotion tasks, learning running gaits for bipedal and quadrupedal simulated robots, and learning a policy for getting the biped to stand up from starting out lying on the ground. In contrast to a body of prior work that uses hand-crafted policy representations, our neural network policies map directly from raw kinematics to joint torques. Our algorithm is fully model-free, and the amount of simulated experi-

Used with TRPO and in PPO to balance bias-variance tradeoff in policy gradient estimates.

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

Naieve advantage estimates:

$$\hat{A}_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} - V(s_t)$$

High variance

Generalized advantage estimation (GAE)

HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel
Department of Electrical Engineering and Computer Science
University of California, Berkeley
{joschu,pcmoritz,levine,jordan,pabbeel}@eecs.berkeley.edu

ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with an exponentially-weighted estimator of the advantage function that is analogous to TD(λ). We address the second challenge by using trust region optimization procedure for both the policy and the value function, which are represented by neural networks.

Our approach yields strong empirical results on highly challenging 3D locomotion tasks, learning running gaits for bipedal and quadrupedal simulated robots, and learning a policy for getting the biped to stand up from starting out lying on the ground. In contrast to a body of prior work that uses hand-crafted policy representations, our neural network policies map directly from raw kinematics to joint torques. Our algorithm is fully model-free, and the amount of simulated experi-

Used with TRPO and in PPO to balance bias-variance tradeoff in policy gradient estimates.

Naieve advantage estimates:

$$\hat{A}_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} - V(s_t)$$

High variance

TD-error one-step estimates:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

High bias, lower variance.

Generalized advantage estimation (GAE)

HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel
Department of Electrical Engineering and Computer Science
University of California, Berkeley
{joschu,pcmoritz,levine,jordan,pabbeel}@eecs.berkeley.edu

ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with an exponentially-weighted estimator of the advantage function that is analogous to TD(λ). We address the second challenge by using trust region optimization procedure for both the policy and the value function, which are represented by neural networks.

Our approach yields strong empirical results on highly challenging 3D locomotion tasks, learning running gaits for bipedal and quadrupedal simulated robots, and learning a policy for getting the biped to stand up from starting out lying on the ground. In contrast to a body of prior work that uses hand-crafted policy representations, our neural network policies map directly from raw kinematics to joint torques. Our algorithm is fully model-free, and the amount of simulated experi-

Used with TRPO and in PPO to balance bias-variance tradeoff in policy gradient estimates.

Naieve advantage estimates:

$$\hat{A}_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} - V(s_t)$$

High variance

TD-error one-step estimates:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

High bias, lower variance.

GAE: trade off between the two with a single parameter λ.

Generalized advantage estimation (GAE)

HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel
Department of Electrical Engineering and Computer Science
University of California, Berkeley
{joschu,pcmoritz,levine,jordan,pabbeel}@eecs.berkeley.edu

ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with an exponentially-weighted estimator of the advantage function that is analogous to TD(λ). We address the second challenge by using trust region optimization procedure for both the policy and the value function, which are represented by neural networks.

Our approach yields strong empirical results on highly challenging 3D locomotion tasks, learning running gaits for bipedal and quadrupedal simulated robots, and learning a policy for getting the biped to stand up from starting out lying on the ground. In contrast to a body of prior work that uses hand-crafted policy representations, our neural network policies map directly from raw kinematics to joint torques. Our algorithm is fully model-free, and the amount of simulated experi-

Used with TRPO and in PPO to balance bias-variance tradeoff in policy gradient estimates.

Naieve advantage estimates:

$$\hat{A}_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} - V(s_t)$$

High variance

TD-error one-step estimates:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

High bias, lower variance.

GAE: trade off between the two with a single parameter λ.

Define k-step advantage estimators:

$$\begin{aligned}
\hat{A}_t^{(1)} &:= \delta_t^V &&= -V(s_t) + r_t + \gamma V(s_{t+1}) \\
\hat{A}_t^{(2)} &:= \delta_t^V + \gamma\delta_{t+1}^V &&= -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) \\
\hat{A}_t^{(3)} &:= \delta_t^V + \gamma\delta_{t+1}^V + \gamma^2\delta_{t+2}^V &&= -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3})
\end{aligned}$$

$$\hat{A}_t^{(k)} := \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k})$$

Generalized advantage estimation (GAE)

HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel
Department of Electrical Engineering and Computer Science
University of California, Berkeley
{joschu,pcmoritz,levine,jordan,pabbeel}@eecs.berkeley.edu

ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with an exponentially-weighted estimator of the advantage function that is analogous to TD(λ). We address the second challenge by using trust region optimization procedure for both the policy and the value function, which are represented by neural networks.

Our approach yields strong empirical results on highly challenging 3D locomotion tasks, learning running gaits for bipedal and quadrupedal simulated robots, and learning a policy for getting the biped to stand up from starting out lying on the ground. In contrast to a body of prior work that uses hand-crafted policy representations, our neural network policies map directly from raw kinematics to joint torques. Our algorithm is fully model-free, and the amount of simulated experi-

Used with TRPO and in PPO to balance bias-variance tradeoff in policy gradient estimates.

Naieve advantage estimates:

$$\hat{A}_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} - V(s_t)$$

High variance

TD-error one-step estimates:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

High bias, lower variance.

GAE: trade off between the two with a single parameter λ.

Define k-step advantage estimators:

$$\begin{aligned}
\hat{A}_t^{(1)} &:= \delta_t^V &&= -V(s_t) + r_t + \gamma V(s_{t+1}) \\
\hat{A}_t^{(2)} &:= \delta_t^V + \gamma\delta_{t+1}^V &&= -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) \\
\hat{A}_t^{(3)} &:= \delta_t^V + \gamma\delta_{t+1}^V + \gamma^2\delta_{t+2}^V &&= -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3})
\end{aligned}$$

$$\hat{A}_t^{(k)} := \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k})$$

$GAE(\gamma, \lambda)$ is defined as the exponentially-weighted average of these k-step estimators:

$$\hat{A}_t^{GAE(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+1}^V$$

Generalized advantage estimation (GAE)

HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel
Department of Electrical Engineering and Computer Science
University of California, Berkeley
{joschu,pcmoritz,levine,jordan,pabbeel}@eecs.berkeley.edu

ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with an exponentially-weighted estimator of the advantage function that is analogous to TD(λ). We address the second challenge by using trust region optimization procedure for both the policy and the value function, which are represented by neural networks.

Our approach yields strong empirical results on highly challenging 3D locomotion tasks, learning running gaits for bipedal and quadrupedal simulated robots, and learning a policy for getting the biped to stand up from starting out lying on the ground. In contrast to a body of prior work that uses hand-crafted policy representations, our neural network policies map directly from raw kinematics to joint torques. Our algorithm is fully model-free, and the amount of simulated experi-

Used with TRPO and in PPO to balance bias-variance tradeoff in policy gradient estimates.

Naieve advantage estimates:

$$\hat{A}_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} - V(s_t)$$

High variance

TD-error one-step estimates:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

High bias, lower variance.

GAE: trade off between the two with a single parameter λ.

Define k-step advantage estimators:

$$\begin{aligned}
\hat{A}_t^{(1)} &:= \delta_t^V &&= -V(s_t) + r_t + \gamma V(s_{t+1}) \\
\hat{A}_t^{(2)} &:= \delta_t^V + \gamma\delta_{t+1}^V &&= -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) \\
\hat{A}_t^{(3)} &:= \delta_t^V + \gamma\delta_{t+1}^V + \gamma^2\delta_{t+2}^V &&= -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3})
\end{aligned}$$

$$\hat{A}_t^{(k)} := \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k})$$

$GAE(\gamma, \lambda)$ is defined as the exponentially-weighted average of these k-step estimators:

$$\hat{A}_t^{GAE(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+1}^V$$

$\lambda = 1$: Monte Carlo,
$\lambda = 0$: 1-step TD.

Generalized advantage estimation (GAE)

HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel
Department of Electrical Engineering and Computer Science
University of California, Berkeley
{joschu,pcmoritz,levine,jordan,pabbeel}@eecs.berkeley.edu

ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with an exponentially-weighted estimator of the advantage function that is analogous to TD(λ). We address the second challenge by using trust region optimization procedure for both the policy and the value function, which are represented by neural networks.

Our approach yields strong empirical results on highly challenging 3D locomotion tasks, learning running gaits for bipedal and quadrupedal simulated robots, and learning a policy for getting the biped to stand up from starting out lying on the ground. In contrast to a body of prior work that uses hand-crafted policy representations, our neural network policies map directly from raw kinematics to joint torques. Our algorithm is fully model-free, and the amount of simulated experi-

Used with TRPO and in PPO to balance bias-variance tradeoff in policy gradient estimates.

Naieve advantage estimates:

$$\hat{A}_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} - V(s_t)$$

High variance

TD-error one-step estimates:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

High bias, lower variance.

GAE: trade off between the two with a single parameter λ.

Define k-step advantage estimators:

$$\begin{aligned}
\hat{A}_t^{(1)} &:= \delta_t^V &&= -V(s_t) + r_t + \gamma V(s_{t+1}) \\
\hat{A}_t^{(2)} &:= \delta_t^V + \gamma\delta_{t+1}^V &&= -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) \\
\hat{A}_t^{(3)} &:= \delta_t^V + \gamma\delta_{t+1}^V + \gamma^2\delta_{t+2}^V &&= -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3})
\end{aligned}$$

$$\hat{A}_t^{(k)} := \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k})$$

$GAE(\gamma, \lambda)$ is defined as the exponentially-weighted average of these k-step estimators:

$$\hat{A}_t^{GAE(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+1}^V$$

$\lambda = 1$: Monte Carlo,
$\lambda = 0$: 1-step TD.

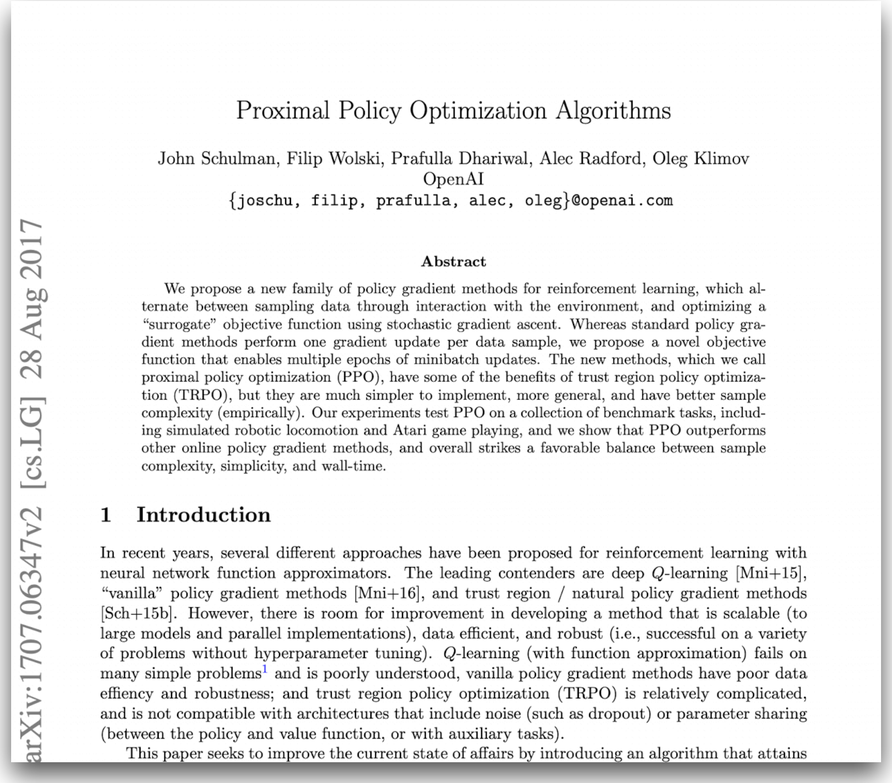
Greatly reduces gradient noise, yielding faster, more stable policy updates.

Proximal policy optimization (PPO)

Key idea: keep TRPO's core feature (don't let the policy move too far from old one), but replace the constrained optimization, with a simple, first-order clipping approach.

Instead of solving a constrained optimization like TRPO, PPO modifies the objective function itself, so that gradient-based optimization methods naturally stay within a trust region.

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)} \hat{A}_t, \text{clip} \left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon \right) \hat{A}_t \right) \right]$$



arXiv:1707.06347v2 [cs.LG] 28 Aug 2017

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a "surrogate" objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q-learning [Mni+15], "vanilla" policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q-learning (with function approximation) fails on many simple problems[1] and is poorly understood, vanilla policy gradient methods have poor data effiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

This paper seeks to improve the current state of affairs by introducing an algorithm that attains

The primary PPO objective

TRPO maximizes a surrogate objective $L^{CPI}(\theta) = \mathbb{E}_t \left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)} \hat{A}_t \right]$ subject to constraints on the total variation divergence between the distributions of the new and old policies.

The primary PPO objective

TRPO maximizes a surrogate objective $L^{CPI}(\theta) = \mathbb{E}_t \left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)} \hat{A}_t \right]$ subject to constraints on the total variation divergence between the distributions of the new and old policies.

Instead of solving such a constrained optimization problem, PPO modifies the TRPO surrogate objective with a “clip” term, to prevent the ratio $\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}$ from moving outside the interval $[1-\epsilon, 1+\epsilon]$.

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)} \hat{A}_t, \text{clip} \left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon \right) \hat{A}_t \right) \right]$$

The primary PPO objective

TRPO maximizes a surrogate objective $L^{CPI}(\theta) = \mathbb{E}_t \left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)} \hat{A}_t \right]$ subject to constraints on the total variation divergence between the distributions of the new and old policies.

Instead of solving such a constrained optimization problem, PPO modifies the TRPO surrogate objective with a “clip” term, to prevent the ratio $\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}$ from moving outside the interval $[1-\epsilon, 1+\epsilon]$.

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)} \hat{A}_t, \text{clip} \left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon \right) \hat{A}_t \right) \right]$$

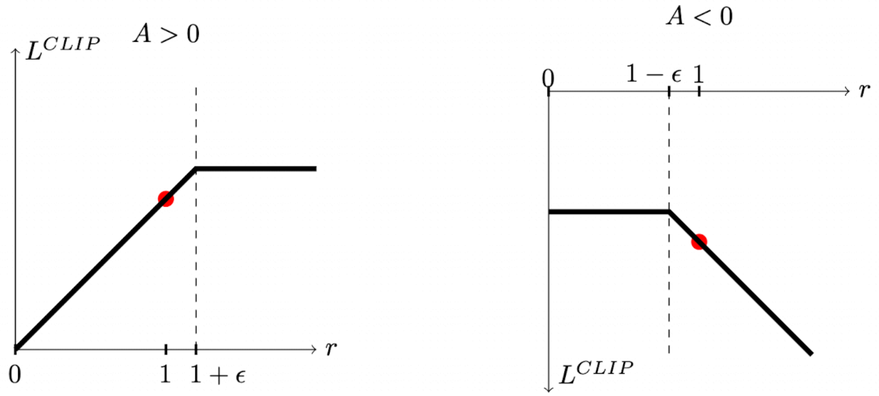


Figure 1: Plots showing one term (i.e., a single timestep) of the surrogate function L^{CLIP} as a function of the probability ratio r, for positive advantages (left) and negative advantages (right). The red circle on each plot shows the starting point for the optimization, i.e., $r = 1$. Note that L^{CLIP} sums many of these terms.

The primary PPO objective

TRPO maximizes a surrogate objective $L^{CPI}(\theta) = \mathbb{E}_t \left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)} \hat{A}_t \right]$ subject to constraints on the total variation divergence between the distributions of the new and old policies.

Instead of solving such a constrained optimization problem, PPO modifies the TRPO surrogate objective with a “clip” term, to prevent the ratio $\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}$ from moving outside the interval $[1-\epsilon, 1+\epsilon]$.

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)} \hat{A}_t, \text{clip} \left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon \right) \hat{A}_t \right) \right]$$

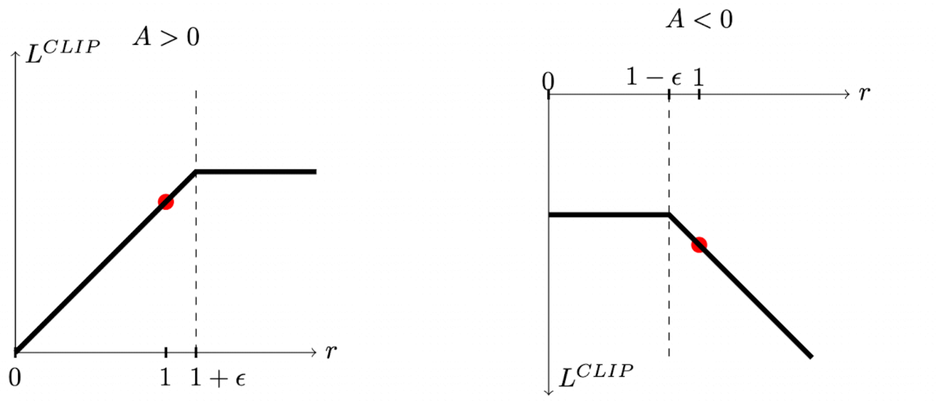


Figure 1: Plots showing one term (i.e., a single timestep) of the surrogate function L^{CLIP} as a function of the probability ratio r, for positive advantages (left) and negative advantages (right). The red circle on each plot shows the starting point for the optimization, i.e., $r = 1$. Note that L^{CLIP} sums many of these terms.

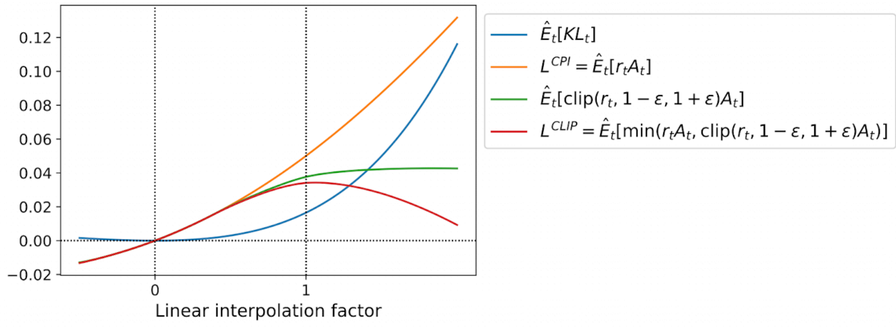


Figure 2: Surrogate objectives, as we interpolate between the initial policy parameter θ_{old}, and the updated policy parameter, which we compute after one iteration of PPO. The updated policy has a KL divergence of about 0.02 from the initial policy, and this is the point at which L^{CLIP} is maximal. This plot corresponds to the first policy update on the Hopper-v1 problem, using hyperparameters provided in Section 6.1.

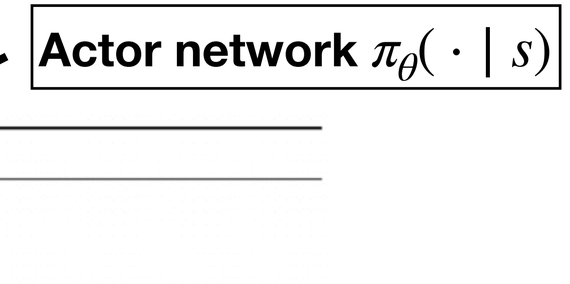
The PPO algorithm

The PPO algorithm

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

The PPO algorithm



Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

The PPO algorithm

Actor network $\pi_\theta(\cdot \mid s)$

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

Use truncated GAE

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \ldots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$$

Where $\delta_t = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)$ using **critic network** $V_\theta(s)$.

The PPO algorithm

Actor network $\pi_\theta(\cdot \mid s)$

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

Use truncated GAE

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \ldots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$$

Where $\delta_t = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)$ using **critic network** $V_\theta(s)$.

$$L_t^{PPO}(\theta) = \mathbb{E}_t\left[\min\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_\theta^{old}(a_t \mid s_t)}\hat{A}_t, \text{clip}\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_\theta^{old}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right]$$
$$-c_1\left(V_\theta(s_t) - V_t^{targ}\right)^2 + c_2\text{entropy}(\pi_\theta(s_t))$$

PPO experimental results

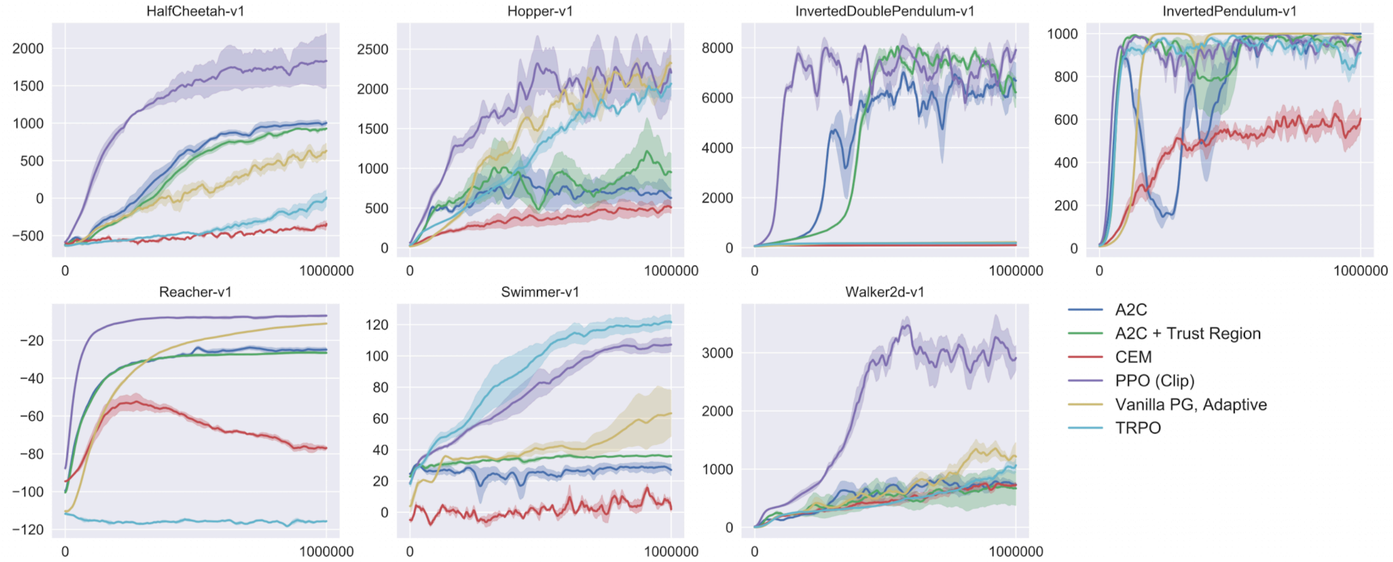
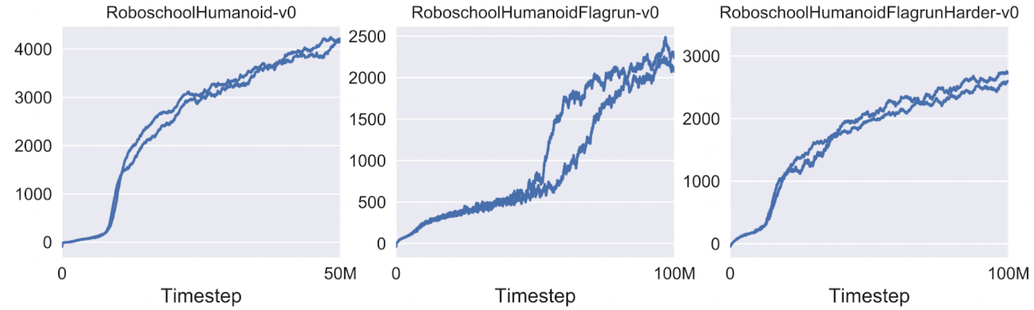


Figure 3: Comparison of several algorithms on several MuJoCo environments, training for one million timesteps.



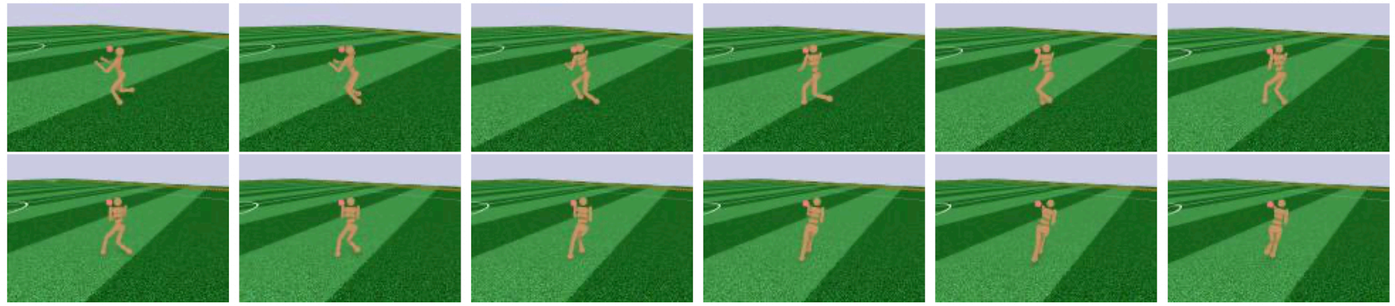


Figure 5: Still frames of the policy learned from RoboschoolHumanoidFlagrun. In the first six frames, the robot runs towards a target. Then the position is randomly changed, and the robot turns and runs toward the new target.

Figure 4: Learning curves from PPO on 3D humanoid control tasks, using Roboschool.

What *is* PPO, conceptually?

- Algorithm family: on-policy actor critic method.
  - ↳ use rollouts from current policy and a learned value function to take gradient steps.
- Theoretical lineage: Approximate trust region methods
  - → clip term limits the change in the policy
- optimization: use SGD.

What are the strengths and weaknesses of PPO?

strengths

- Super simple to use. Just SGD for optimization.
- Relatively stable learning
- Good empirical performance, easy to parallelize.
- use it! when easy to collect large batches of data, or when you just want something that works.

weaknesses

- Sample inefficient. Still on-policy at end of the day.
- Hyperparameter sensitivity.
- Don't use it: when off-policy data re-use would help, you need data efficiency.